

TP 1 - ORM con Hibernate y anotaciones JPA

Descripción del Dominio

Se quiere generar una app de viajes turísticos para un emprendimiento local.

Una vez que una persona se ha registrado en el sistema de Turismo, puede elegir alguno de los varios recorridos diarios que la empresa ofrece y comprarlo.

Cada recorrido (Route) tiene su nombre, que es representativo del tramo que visita, el valor del mismo, el total en km del recorrido y pueden visualizarse además cada una de las paradas (Stop) que componen el recorrido. Todos los recorridos se realizan una vez por día. Asimismo, la aplicación registra comentarios y valoraciones (estrellas) de otros usuarios que han realizado el mismo recorrido.

Dada la cantidad de personas que solicitan los recorridos, la empresa ha tenido que limitar cada recorrido a un tope máximo de personas. Esa cantidad no es general sino que se define para cada uno y se deben vender solo mientras el cupo lo permita.

Adicionalmente, se conoce un grupo de choferes y guías turísticos que participan del recorrido.

A modo de ejemplo, podemos pensar en el recorrido "Tour tradicional de Bs. As." que cuenta con paradas en distintos lugares emblemáticos de la ciudad de Buenos Aires como Caminito en La Boca, San Telmo, Obelisco, lago de Palermo y Recoleta; o en "Tour del Delta" que incluye a San Isidro, el Delta y también al Obelisco.

Una vez que se ha registrado la compra de un usuario para un recorrido determinado (Purchase), se le habilita la posibilidad de agregar comentarios y puntaje al recorrido realizado (Review) de forma tal que luego puedan usarlo en promociones del emprendimiento y poder captar así a más público interesado en los mismos.

Se ha pensado que los usuarios (User) se identifiquen en la aplicación mediante usuario y contraseña, y otros datos que pueden resultar útiles para posteriores ofertas de tours que la empresa realice. Por eso se les solicitan datos personales y de contacto como el nombre, un número de celular, un email, edad y fecha de nacimiento.

Los choferes (DriverUser) y guías turísticos (TourGuideUser) también son usuarios del sistema, por lo que se registran los mismos datos que los usuarios, pero además cuentan con una serie de antecedentes que la empresa quiere mantener: del chofer, de trabajos anteriores; del guía turístico, sobre su formación en turismo. Cabe mencionar que los choferes y los guías no están afectados a un recorrido en particular, sino que pueden participar de varios según la demanda y

la necesidad de la empresa. En fechas particulares, por ejemplo, hay mayor demanda por los tours de la zona del Delta.

Durante todo el recorrido se ofrecen diversos servicios (Service) que un usuario puede contratar. De los servicios se conoce su nombre, su valor, una descripción y el proveedor que lo brinda. Por ejemplo, la confección de algún recuerdo (un mate pintado artesanalmente, una remera o una foto en un lugar determinado).

De cada proveedor (Supplier) se conocen los servicios que ofrece, la razón social y un número de habilitación provincial.

Cada ítem de servicio aparece en la compra junto con el recorrido y la cantidad solicitada del mismo (ItemService).

La compra final contendrá: el recorrido elegido y todos los servicios solicitados a lo largo del mismo (para que el usuario pueda tener control de lo abonado). Además, sobre cada compra se registra un código interno y la fecha.

Diagrama de Clases del Sistema



Descripción de las Entidades

Entidad	Descripción	Relaciones Principales
Service	Servicio ofrecido con nombre y precio	Pertenece a un Supplier; referenciado por ItemService

Entidad	Descripción	Relaciones Principales
Supplier	Proveedor que ofrece servicios	Tiene muchos Service
ItemService	Item de servicio en una compra (cantidad)	Referencia a Service; pertenece a Purchase
Purchase	Compra de un usuario sobre una ruta	Tiene ItemService, User, Route y Review (0..1)
Review	Reseña con rating y comentario	Asociada a una Purchase (0..1)
User	Usuario base del sistema	Subclasificado en DriverUser y TourGuideUser
DriverUser	Conductor (subclase de User)	Asignado a una o más Route como driver
TourGuideUser	Guia turistico (subclase de User)	Asignado a una o más Route como tourGuide
Route	Ruta con nombre, precio y capacidad	Tiene Stop (1..*), drivers y tourGuides
Stop	Parada de una ruta	Composicion con Route (1..*)

Sección 1 - Introducción a los ORM

1.1 El problema del ORM

1. ¿Qué problema concreto resuelve un ORM? Describir al menos 3 (tres) inconvenientes que aparecen al intentar persistir objetos directamente con JDBC puro.

Un **ORM (Object-Relational Mapper)** busca resolver, fundamentalmente, la **Diferencia de Impedancia (Impedance Mismatch)**. Este problema es la desconexión estructural y conceptual entre el modelo de objetos y el modelo relacional.

Algunos inconvenientes:

- *"Contaminación" del código y dificultad de mantenimiento*
 - Al usar JDBC, el desarrollador debe escribir manualmente cadenas de texto (Strings) con sentencias SQL dentro del código de la aplicación. Esto genera una fuerte dependencia del motor de base de datos específico.
- *Gestión manual de la jerarquía e identidad*
 - El modelo relacional no soporta de forma nativa la **herencia** ni el **polimorfismo**, conceptos centrales en la Programación Orientada a Objetos. Con JDBC, el

programador debe decidir y programar manualmente cómo transformar una jerarquía de clases en tablas.

- *Complejidad en la carga de grafos de objetos (Lazy Loading)*
 - En un sistema OO, los objetos están interconectados en grafos que pueden ser muy extensos. Con JDBC, al recuperar un objeto, el programador debe decidir mediante código explícito si trae también todos sus objetos relacionados. Si trae todo el grafo "en cascada", puede agotar la memoria; si no lo trae, debe realizar múltiples consultas adicionales a la base de datos cada vez que necesite un dato relacionado.
2. El modelo relacional y el modelo OO presentan tensiones conocidas como impedance mismatch. Identificar cómo se manifiesta cada una en el modelo dado:
1. Identidad: cómo es posible identificar un objeto Purchase en Java vs. en la base de datos?
 2. Relaciones: cómo se navega de una Purchase a sus ItemService en Java vs. en SQL?
 3. Herencia: ¿cómo podría representarse la jerarquía User/DriverUser/TourGuideUser en una tabla relacional?
 4. Ciclos de referencia: ¿existe algún ciclo en el modelo? ¿Cómo impacta en la persistencia?

Identidad: cómo es posible identificar un objeto Purchase en Java vs. en la base de datos?

- En Java un objeto `Purchase` se identifica por su referencia en memoria mientras que en una base de datos los hace por una clave primaria. Esto quiere decir que en Java podemos tener dos objetos iguales pero distintos mientras que en la db no.

Relaciones: cómo se navega de una Purchase a sus ItemService en Java vs. en SQL?

- En Java navegamos directamente con referencias como por ejem. `purchase.getItems(); item.getService();` mientras que en SQL se tiene que hacer mediante joins tipo:

```
SELECT *  
FROM purchase p  
JOIN item_service i on i.purchase_id = p.id  
JOIN service s ON s.id = i.service_id
```

Herencia: ¿cómo podría representarse la jerarquía User/DriverUser/TourGuideUser en una tabla relacional?

- En Java se puede hacer de manera directa (código abajo), mientras que en SQL no existe la Herencia, se podría dejar toda la información en una tabla teniendo atributos null, o hacer una tabla por clase (lo que hace que los joins sean más complejos), o usar *JOINED* de Hibernate.

```
class User {}  
class DriverUser extends User {}  
class TourGuideUser extends User {}
```

Ciclos de referencia: ¿existe algún ciclo en el modelo? ¿Cómo impacta en la persistencia?

- Si, hay ciclos de referencia, por ejemplo está el de `Purchase` y `User`. En la persistencia afecta en:
 - *Orden de guardado*: La `Purchase` necesita el `User` y el `User` tiene a la `Purchase` también, no se define cuál guardar primero.
 - *Cascadas*: Se pueden generar inserts duplicados en la db.
3. Describir las ventajas y desventajas concretas de usar un ORM en un proyecto como este.

Ventajas de usar un ORM en este proyecto

1. **Simplificación de la Jerarquía de Usuarios**: El modelo presenta una estructura de herencia clara: `User` como superclase y `DriverUser` y `TourGuideUser` como subclases. Un ORM permite mapear esta jerarquía de forma que el desarrollador pueda realizar consultas polimórficas de manera transparente.
2. **Gestión Transparente de Relaciones Complejas**: El proyecto tiene relaciones de varios tipos: composición (`Route` -> `Stop`), uno a muchos (`Purchase` -> `ItemService`) y muchos a muchos (`Route` <-> `DriverUser`). El ORM permite navegar estas relaciones mediante código Java puro en lugar de gestionar manualmente claves foráneas y tablas intermedias en cada consulta.
3. **Persistencia por Alcance (Reachability)**: En este dominio, una `Purchase` (Compra) contiene una lista de `ItemService`. Gracias a la **persistencia por alcance**, al guardar o actualizar una compra, el ORM puede propagar automáticamente los cambios a todos sus ítems relacionados, reduciendo drásticamente la cantidad de llamadas explícitas a "save" en los repositorios.
4. **Optimización mediante Lazy Loading**: Las rutas pueden tener muchas paradas (`Stop`) y los usuarios muchas compras. El uso de **Lazy Loading** asegura que estos grafos de objetos no se carguen en memoria a menos que sean estrictamente necesarios, mejorando la performance y el uso de recursos del servidor.
5. **Productividad y Mantenibilidad**: Al automatizar el mapeo de los **POJOs** (objetos Java simples), el equipo se enfoca en la lógica de negocio y no en el "trabajo pesado" de traducir datos entre tablas y objetos.

Desventajas y Desafíos del uso de un ORM

1. **"Contaminación" de las Clases de Dominio:** Si se utilizan **anotaciones JPA** (como `@Entity` o `@ManyToOne`), se rompe el principio de **Independencia**. Las clases del modelo quedan ligadas a metadatos de persistencia, lo que "ensucia" el código que debería ser puramente de negocio.
2. **Curva de Aprendizaje del Ciclo de Vida:** Hibernate gestiona las entidades a través de estados (`transient`, `managed`, `detached`, `removed`). Si el desarrollador no comprende bien estos estados, pueden surgir errores al intentar modificar objetos que ya no están asociados a una sesión activa.
3. **Abstracción "Fugadiza" (Leaky Abstraction):** Aunque el ORM busca ocultar la base de datos, no es perfecto. Si se realizan operaciones masivas o consultas muy ineficientes sobre el grafo de objetos, el rendimiento caerá drásticamente, obligando al desarrollador a conocer el SQL generado y a realizar ajustes técnicos manuales.
4. **Riesgos de la Persistencia en Cascada:** Si no se configuran correctamente las operaciones en cascada, se podrían eliminar datos maestros de forma accidental por ejemplo.
5. **Complejidad en el Control de Concurrency:** En un sistema de viajes donde varios usuarios pueden comprar el mismo recorrido simultáneamente, el manejo de **bloqueos optimistas** (usando la anotación `@Version`) es vital. Configurar y manejar estas excepciones de concurrencia añade una capa de complejidad al código de los servicios.

1.2 JPA e Hibernate

4. ¿Qué es JPA? ¿Qué nos permite definir? ¿Por qué se dice que es una especificación y no una implementación?

JPA es el **estándar o especificación oficial de persistencia** para la plataforma Java. Se define como un **contrato de persistencia estándar** que establece un conjunto de reglas y definiciones para gestionar cómo los objetos de una aplicación Java se almacenan y recuperan de una base de datos.

Proporciona una **API y un conjunto de anotaciones** que permiten a los desarrolladores describir el mapeo entre el modelo de objetos y el modelo relacional de forma declarativa. A través de JPA se pueden definir:

- **Entidades y Metadatos:** Cuáles clases son persistentes (usando `@Entity`) y cuál es su identidad única (usando `@Id`).
- **Estrategias de Mapeo de Jerarquías:** Cómo transformar la herencia de objetos en tablas (como *Single Table*, *Joined* o *Table per Class*).
- **Relaciones entre objetos:** Definir asociaciones de tipo Uno a Uno, Uno a Muchos o Muchos a Muchos.
- **Comportamiento de Carga (Fetch Type):** Decidir si los datos relacionados se cargan bajo demanda (**Lazy**) o de forma inmediata (**Eager**).

- **Operaciones en Cascada:** Determinar qué operaciones (guardar, borrar, etc.) se deben propagar automáticamente a los objetos relacionados, concepto conocido como **Persistencia por Alcance**.
- **Consultas:** Utilizar un lenguaje de consultas orientado a objetos denominado **JPQL** (Java Persistence Query Language)

Se dice que es una **especificación** porque JPA solo define el **"qué"** (las interfaces, anotaciones y reglas del contrato), pero no proporciona el código ejecutable que realiza el trabajo pesado de conectarse a la base de datos y ejecutar el SQL, para que JPA funcione, necesita de una **implementación concreta** que traduzca esas definiciones en acciones reales.

5. ¿Cuál es la relación entre JPA e Hibernate? ¿Puede usarse JPA sin Hibernate o Hibernate sin JPA?

Esto es maso lo de arriba, la relación es que **JPA** es el **Qué**, te define el contrato, interfaces, anotaciones, etc. y **Hibernate** es el **Cómo**, proporciona las implementaciones concretas de lo que define JPA.

Técnicamente no se puede usar JPA "solo", porque JPA es una especificación y requiere de un **proveedor de persistencia** (Provider) que contenga el código ejecutable. Sin embargo, **sí se puede usar JPA sin Hibernate específicamente**, siempre que se utilice otra implementación que cumpla con el estándar.

Hibernate sin JPA obvio que se puede usar, es más, Hibernate existía mucho antes de que se creara el estándar JPA.

6. ¿Qué es la SessionFactory? ¿Qué patrón de diseño implementa? Justificar por qué se crea una sola instancia durante todo el ciclo de vida de la aplicación y no una por operación

La **SessionFactory** es uno de los componentes centrales de la arquitectura de Hibernate. Se define como un objeto **inmutable que actúa como un mapa o caché de mapeos compilados** para una base de datos específica.

La **SessionFactory** implementa principalmente **dos patrones de diseño**:

1. **Factory:** Como su nombre lo indica, su función principal es actuar como una fábrica para abrir y proveer instancias de objetos **Session** (sesiones de trabajo) a la aplicación.
2. **Singleton:** Aunque Hibernate no obliga estrictamente a usarlo como Singleton por código, en la práctica se maneja como tal dentro del ciclo de vida de la aplicación. Existe una única instancia para cada configuración de base de datos para centralizar los recursos.

Existen razones técnicas y de performance fundamentales para mantener una única instancia durante todo el ciclo de vida de la aplicación:

- **Alto costo de inicialización (Heavyweight object):** La creación de una `SessionFactory` es una operación extremadamente costosa en términos de tiempo y procesamiento. Al inicializarse, Hibernate debe leer todos los archivos de configuración (como `hibernate.cfg.xml`), parsear los metadatos de mapeo (`.hbm.xml` o anotaciones), validarlos y compilarlos en una estructura interna eficiente.
- **Gestión del Second Level Cache (Caché de Segundo Nivel):** La `SessionFactory` es responsable de mantener la caché de segundo nivel, la cual es compartida por todas las sesiones de la aplicación. Si se creara una instancia por operación, no existiría un lugar común para almacenar objetos recuperados frecuentemente, perdiendo la capacidad de optimizar los accesos a la base de datos a nivel global.
- **Seguridad de hilos (Thread-Safety):** Al ser un objeto inmutable una vez compilado, es inherentemente **seguro para ser compartido entre múltiples hilos** de ejecución. Esto permite que toda la aplicación le pida sesiones de forma concurrente sin riesgos de corrupción de datos en la configuración.
- **Administración de conexiones:** Funciona como el configurador maestro del pool de conexiones hacia la base de datos. Centralizar esto en un solo objeto permite un control eficiente sobre cuántas conexiones se abren y cómo se distribuyen entre las sesiones.

7. La `SessionFactory` ofrece dos formas de obtener una `Session`: `openSession()` y `getCurrentSession()`. Responder:

1. ¿Cuál es la diferencia entre ambos métodos? ¿Qué ocurre con el ciclo de vida de la `Session` en cada caso?
2. ¿Qué condición debe cumplirse para poder usar `getCurrentSession()`? ¿Que configuración requiere en Hibernate?
3. ¿Quién es responsable de cerrar la `Session` cuando se usa `getCurrentSession()`? ¿Y cuando se usa `openSession()`?
4. ¿Cuál de los dos métodos resulta más adecuado para usar en los repositorios de esta práctica y por qué?

¿Cuál es la diferencia entre ambos métodos? ¿Qué ocurre con el ciclo de vida de la `Session` en cada caso?

<code>openSession()</code>	<code>getCurrentSession()</code>
Este método siempre abre una nueva sesión en la base de datos, independientemente de si ya existe una activa en el contexto actual. El desarrollador tiene el control total sobre su ciclo de vida: debe abrirla, iniciar la transacción, confirmar los cambios y,	Este método busca una sesión que ya esté vinculada al contexto actual (usualmente asociado a la transacción en curso). Si no existe una, la crea; si ya existe, devuelve la misma instancia. Su ciclo de vida está íntimamente ligado al de la transacción: cuando la transacción termina (ya sea por

<code>openSession()</code>	<code>getCurrentSession()</code>
fundamentalmente, cerrarla manualmente para evitar fugas de memoria	<i>commit</i> o <i>rollback</i>), la sesión se cierra automáticamente

¿Qué condición debe cumplirse para poder usar `getCurrentSession()`? ¿Que configuración requiere en Hibernate?

- Para poder utilizar `getCurrentSession()`, debe cumplirse la condición de que **Hibernate** sepa cómo gestionar el "contexto" de la sesión actual (y que haya una sesión, medio complicado si no la hay).
 - Configuración:** Requiere definir la propiedad `hibernate.current_session_context_class` en el archivo de configuración (`hibernate.cfg.xml`).

¿Quién es responsable de cerrar la Session cuando se usa `getCurrentSession()`? ¿Y cuando se usa `openSession()`?

Cerrar sesión con <code>openSession()</code>	Cerrar sesión con <code>getCurrentSession()</code>
La responsabilidad recae enteramente en el desarrollador . Es obligatorio incluir una llamada a <code>session.close()</code> para liberar la conexión al pool	La responsabilidad es de Hibernate (o del framework de soporte como Spring). Al finalizar la unidad de trabajo (la transacción), el motor detecta el cierre y libera los recursos de forma automática, reduciendo errores humanos

¿Cuál de los dos métodos resulta más adecuado para usar en los repositorios de esta práctica y por qué?

- El método más adecuado para esta práctica es `getCurrentSession()` (si querés hilar fino en la sección 3.1 punto 41. lo dice explícitamente...)
 - Gestión de Transacciones:** Al usar este método, se garantiza que todas las operaciones realizadas dentro de un mismo hilo o transacción (coordinadas por la capa de Servicio) compartan el mismo **Persistence Context** y la misma **Caché de Nivel 1**.
 - Simplicidad:** Evita tener que pasar el objeto `Session` como parámetro entre métodos del repositorio o gestionar manualmente el cierre de conexiones en cada operación de acceso a datos.

8. Completar la tabla comparativa entre JPA e Hibernate:

Aspecto	JPA	Hibernate
Tipo	Especificación	Implementación

Aspecto	JPA	Hibernate
Define interfaces y anotaciones	Si	Si (Implementa las de JPA y añade anotaciones propias adicionales)
Proporciona implementaciones concretas	No	Si
Genera SQL a partir de consultas JPQL/HQL	No	Si
Maneja el Persistence Context	Define el contrato y comportamiento esperado del contexto	Lo gestiona físicamente (usualmente mediante el objeto <code>Session</code>)

1.3 Ciclo de vida de las entidades

9. Describir los cuatro estados posibles de una entidad en Hibernate (transient, managed, detached, removed) e indicar qué operación dispara cada transición entre ellos.

Transient: El objeto ha sido instanciado recientemente en la memoria de la aplicación (usualmente con el operador `new`), pero aún no tiene una identidad en la base de datos (Clave Primaria) ni está asociado a ninguna sesión de Hibernate

Managed / Persistent: La entidad tiene una identidad en la base de datos (ID) y está asociada activamente a una `Session`. Cualquier cambio realizado en los atributos del objeto será detectado automáticamente por Hibernate y sincronizado con la base de datos al realizar el *flush* o el *commit* de la transacción.

Removed: La entidad tiene una identidad persistente y está asociada a una sesión, pero ha sido marcada para ser eliminada de la base de datos en cuanto se complete la unidad de trabajo.

10. Describir el ciclo de vida completo de una entidad persistente, tome como ejemplo un objeto de la clase `Purchase`: desde que se instancia con `new`, pasando por su persistencia, hasta que se elimina. Indicar explícitamente en qué estado se encuentra el objeto en cada paso.

Ciclo de vida:

- **Estado: Transient:** El ciclo comienza cuando instanciamos el objeto en la memoria de la aplicación.
 1. **Acción:** `Purchase miCompra = new Purchase();`
- **Estado: Managed o Persistent:** El objeto entra en este estado cuando se vincula a una sesión activa de Hibernate.

1. **Acción:** `session.save(miCompra);` o `session.persist(miCompra);`

- **Estado: Detached:** Ocurre cuando el objeto sigue existiendo en memoria pero se rompe su vínculo con la sesión.

1. **Acción:** `session.close();`, `session.clear();` o al finalizar la transacción.

- **Estado: Removed:** El objeto es marcado para ser borrado físicamente del almacenamiento.

1. **Acción:** `session.delete(miCompra);` o `session.remove(miCompra);`

11. Investigue sobre los métodos de Session: `session.save()`, `session.persist()`, `session.merge()` y `session.saveOrUpdate()`. ¿Qué permite hacer cada uno y cuál es la diferencia entre ellos? Indicar en qué estado debe estar un objeto para usar cada uno correctamente.

Descripción de los Métodos de Session:

- `session.persist(Object entity)` : Es el método definido por la **especificación JPA**.
 - **Función:** Marca una instancia transitoria como persistente.
 - **Estado requerido:** El objeto debe estar en estado **Transient**. Si el objeto ya es persistente, no hace nada; si está en estado **Detached**, lanzará una excepción.
 - **Comportamiento:** No garantiza la asignación inmediata del identificador si la estrategia es **IDENTITY**, ya que el **INSERT** puede retrasarse hasta el momento del **flush**. No devuelve nada (`void`).
- `session.save(Object entity)` : Es un método **nativo de Hibernate** que existía antes del estándar JPA.
 - **Función:** Asegura que el objeto se guarde en la base de datos y le asigna un identificador.
 - **Estado requerido:** Principalmente para objetos **Transient**.
 - **Comportamiento:** A diferencia de `persist`, **devuelve el identificador (Serializable)** del objeto de forma inmediata. Si se invoca sobre un objeto que ya tiene ID (Detached), podría intentar un nuevo **INSERT** y generar una excepción de clave duplicada.
- `session.saveOrUpdate(Object entity)` : Es un método **nativo y específico de Hibernate** que busca simplificar el flujo de trabajo.
 - **Función:** Es una operación "inteligente". Si el objeto es nuevo (Transient), ejecuta un `save()` ; si el objeto ya existía y fue modificado fuera de la sesión (Detached), ejecuta un `update()` .
 - **Estado requerido:** Funciona con objetos **Transient** o **Detached**.
 - **Comportamiento:** Hibernate decide qué acción tomar basándose en la presencia o ausencia de un valor en el campo `@Id` o en la versión del objeto
- `session.merge(Object entity)` : Es el método estándar de **JPA** para lidiar con objetos desasociados.

- **Función:** Copia el estado de un objeto dado en una instancia persistente con el mismo identificador.
 - **Estado requerido:** Ideal para objetos en estado **Detached**.
 - **Comportamiento:** Es muy distinto a los anteriores: **devuelve una nueva instancia** que está en estado **Managed**. El objeto original que pasamos como parámetro **sigue estando desasociado (Detached)**; solo la copia devuelta por el método es rastreada por Hibernate.
-

Sección 2 - Mapeo de Entidades

2.1 Mapeo de una entidad simple: Service

12. ¿Cuál es el conjunto mínimo de anotaciones que debe tener una clase para ser persistente con JPA?

Anotaciones Obligatorias:

- **@Entity:** Se coloca a nivel de clase para indicar que esta es una entidad persistente y que sus instancias deben ser almacenadas en la base de datos.
- **@Id:** Se debe aplicar sobre el atributo que actúa como identificador único o clave primaria de la clase, permitiendo al motor de persistencia mapear la identidad del objeto con el registro correspondiente en la tabla.

Si bien esas son las únicas anotaciones obligatorias, la clase también debe cumplir algunas **reglas estructurales de los POJOs**:

- **Constructor sin argumentos:** La clase debe tener obligatoriamente un constructor por defecto (sin parámetros) para que el framework pueda instanciarla al recuperar datos,.
- **Campos No Finales:** La clase y sus métodos persistentes no deben ser declarados como `final`.
- **Convención sobre Configuración:** Anotaciones como `@Table` o `@Column` son opcionales; si se omiten, JPA asume por convención que el nombre de la tabla es el de la clase y el de las columnas el de los atributos,.
- **Generación de IDs:** Aunque no es estrictamente mínima, suele acompañarse la anotación `@Id` con `@GeneratedValue` para que el motor gestione automáticamente el incremento de la clave primaria.

13. ¿Qué significa que JPA use persistencia por alcance (persistence by reachability)? ¿Qué consecuencia tiene si un objeto referenciado no está todavía persistido?

La **Persistencia por Alcance** es un principio que establece que **todo objeto al que se pueda llegar (navegar) a partir de un objeto que ya es persistente, debe volverse persistente a su vez.**

Si un objeto persistente hace referencia a un objeto que todavía es transitorio (`transient`), pueden ocurrir dos escenarios dependiendo de la configuración:

- **Persistencia Automática (Cascada Activa):** Si la relación está configurada con operaciones en cascada, Hibernate **persistirá automáticamente el objeto referenciado.**
- **Error de Persistencia (Cascada Desactivada):** Por defecto, en JPA **no se propaga ninguna operación.** Si intentas confirmar una transacción donde un objeto persistente apunta a uno transitorio y no hay cascada configurada, el sistema fallará (usualmente con una `TransientPropertyValueException`), ya que la base de datos no puede mantener la integridad referencial (FK) hacia un registro que no existe.

14. ¿Qué diferencia hay entre las estrategias IDENTITY, SEQUENCE y TABLE para la generación de IDs? ¿Cuál tiene mejor rendimiento en inserciones masivas y por qué?

Estrategia IDENTITY

- Esta estrategia delega la generación del ID totalmente al motor de la base de datos, utilizando columnas de tipo "auto-increment"
- **Funcionamiento:** El ID se genera solo después de que se ejecuta efectivamente la sentencia `INSERT` en la base de datos.
- **Inconveniente técnico:** Hibernate necesita conocer el ID del objeto para pasarlo del estado *Transient* al estado *Managed* y rastrear sus cambios en el **Persistence Context**. Como el ID solo existe tras el insert, Hibernate se ve obligado a ejecutar la sentencia inmediatamente.
- **Impacto:** Esto **desactiva el procesamiento por lotes (batching)**, ya que Hibernate no puede encolar los inserts para enviarlos juntos.

Estrategia SEQUENCE

- Utiliza un objeto de base de datos específico llamado **Secuencia** para obtener los valores numéricos antes de realizar el insert
- **Funcionamiento:** Hibernate solicita a la base de datos uno o varios valores de la secuencia antes de ejecutar cualquier operación de escritura.
- **Ventaja:** Al obtener el ID de antemano, Hibernate puede asignar la identidad al objeto en memoria y mantenerlo en el **Persistence Context** sin necesidad de disparar el `INSERT` de forma inmediata.
- **Optimización:** Permite el uso de parámetros como `allocationSize` , lo que permite "reservar" un rango de IDs (ej. de 50 en 50) con una sola llamada a la base de datos, minimizando el tráfico de red.

Estrategia TABLE

- Crea una tabla dedicada en el esquema de la base de datos para simular una secuencia, almacenando el último ID generado en una fila.
- **Funcionamiento:** Cada vez que se requiere un ID, el framework lee el valor de la tabla, lo incrementa y actualiza la fila.
- **Ventaja:** Es la estrategia más portable, ya que funciona en cualquier motor de base de datos relacional, incluso en aquellos que no soportan ni identidad ni secuencias nativas.
- **Desventaja:** Tiene un costo de rendimiento elevado debido a que cada solicitud de ID requiere bloqueos (locks) sobre la fila de la tabla de IDs para garantizar la unicidad, lo que genera cuellos de botella en ambientes concurrentes.

La estrategia con **mejor rendimiento para inserciones masivas es SEQUENCE** ya que tiene:

- **Soporte de Batching** permitiendo que el framework acumule múltiples sentencias `INSERT` en memoria y las envíe a la base de datos en un **único paquete o lote (Batch)**, reduciendo drásticamente los viajes de ida y vuelta (roundtrips) al servidor.
- **Pre-asignación de IDs:** mediante el uso de secuencias con incrementos mayores a 1.
- **Menos contención que TABLE** ya que mientras que la estrategia `TABLE` requiere bloqueos físicos en disco sobre una tabla de control, las secuencias están optimizadas a nivel de motor de base de datos para manejar concurrencia de forma mucho más ágil.

15. Implementar el mapeo completo de la entidad Service según el diagrama. La implementación debe incluir:

1. Clave primaria con estrategia de generación automática. Elegir entre `IDENTITY`, `SEQUENCE` o `TABLE` y justificar la elección.
2. Atributos: name (no nulo, max. 100 caracteres), description (opcional), price (no nulo).
3. Al menos una restricción de unicidad a nivel de columna.

```
@Entity
@Table(
    name = "services",
    uniqueConstraints = {
        @UniqueConstraint(columnNames = "name")
    }
)
public class Service {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"service_seq")
    @SequenceGenerator(name = "service_seq", sequenceName =
"service_sequence", allocationSize = 50)
```

```

private Long id;

@Column(nullable = false, length = 100)
private String name;

@Column(nullable = false)
private float price;

@Column
private String description;

@OneToMany(mappedBy = "service", fetch = FetchType.LAZY)
private List<ItemService> itemServiceList;

@ManyToOne
@JoinColumn(name = "supplier_id")
private Supplier supplier;

// getters y setters de la clase
}

```

Clave primaria con estrategia de generación automática. Elegir entre IDENTITY, SEQUENCE o TABLE y justificar la elección.

- Se elige SEQUENCE por las razones previamente mencionadas:
 - Damos soporte de batching que para el dominio puede venir bien ya que es esperable tener inserciones masivas de servicios por ejem. al cargar el catálogo de un proveedor.
 - Con el `allocationSize = 50` Hibernate reserva 50 IDs de una sola consulta a la secuencia, minimizando los roundtrips a la db.

Al menos una restricción de unicidad a nivel de columna.

- Esta restricción la aplicamos con `uniqueConstraints = @UniqueConstraint(columnNames = "name")`. Se aplica sobre `name` porque en el dominio no tendría sentido que dos servicios del mismo proveedor (o del sistema) tengan el mismo nombre, esto generaría ambigüedad en la contratación durante el recorrido. La restricción se define a nivel de `@Table` para que JPA la genere directamente en el DDL de la base de datos.

2.2 Relaciones entre entidades

16. Para la relación Purchase -> ItemService (composición uno-a-muchos):

1. ¿Qué anotaciones se necesitan en cada lado?
2. ¿Qué columna o tabla aparece en la base de datos para representar esta relación?
¿Qué es mappedBy y en qué lado de la relación va? ¿Qué ocurre si se omite en ambos

lados?

3. ¿Es esta relación bidireccional o unidireccional según el diagrama? ¿Cómo se refleja en el código Java?

En la clase Purchase

```
@OneToMany(mappedBy = "purchase", cascade = CascadeType.ALL, orphanRemoval = true)
private List<ItemService> itemServiceList;
```

En la clase ItemService

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "item_service_seq")
@SequenceGenerator(name = "item_service_seq", sequenceName = "item_service_sequence", allocationSize = 50)
Long id;

@Column(nullable = false)
private int quantity;

@ManyToOne
@JoinColumn(name = "purchase_id", nullable = false)
private Purchase purchase;

@ManyToOne
@JoinColumn(name = "service_id", nullable = false)
private Service service;
```

¿Qué columna o tabla aparece en la base de datos para representar esta relación? ¿Qué es mappedBy y en qué lado de la relación va? ¿Qué ocurre si se omite en ambos lados?

- En la base de datos solo aparece una FK en la tabla `item_service`. No se crea ninguna tabla intermedia.
- `mappedBy` es el parámetro que le dice a JPA cuál es el **lado dueño** de la relación, es decir, qué atributo en la otra clase es el que tiene la FK física en la base de datos.
 - El lado con `@ManyToOne` + `@JoinColumn` → **es el dueño** (tiene la FK en la tabla)
 - El lado con `@OneToMany(mappedBy = ...)` → **es el inverso** (solo para navegar en Java, no genera nada en la BD)
- Si se omite en ambos lados JPA interpreta que hay **dos relaciones independientes** y crea una **tabla intermedia innecesaria** con los IDs de las tablas involucradas.

¿Es esta relación bidireccional o unidireccional según el diagrama? ¿Cómo se refleja en el código Java?

- Mirando el diagrama, la relación es **bidireccional**. Desde `Purchase` podemos navegar a sus ítems, y desde `ItemService` podemos navegar a su compra.

```
// Desde Purchase podemos hacer:  
purchase.getItemServiceList() // → lista de ítems  
  
// Desde ItemService podemos hacer:  
itemService.getPurchase()      // → la compra a la que pertenece
```

- Importante:** en relaciones bidireccionales hay que mantener la consistencia de ambos lados manualmente en Java. JPA no lo hace solo. La forma correcta es tener un método helper en `Purchase` ya que si solo se hace `purchase.getItemServiceList().add(item)` sin setear `item.setPurchase(purchase)`, **Hibernate no va a persistir la FK** correctamente porque el lado dueño (`ItemService`) no fue actualizado.

```
public void addItem(ItemService item) {  
    itemServiceList.add(item);  
    item.setPurchase(this); // ← mantiene el lado dueño sincronizado  
}
```

17. Para las relaciones `Route <-> DriverUser` y `Route <-> TourGuideUser` (muchos-a-muchos):

- ¿Qué anotaciones se usan?
- ¿Qué tabla adicional genera JPA? ¿Qué columnas tiene? Definirla explícitamente usando `@JoinTable`.
- ¿Pueden ambas relaciones compartir la misma tabla join? ¿Por qué?

¿Qué anotaciones se usan?

- En una relación muchos-a-muchos, **ninguno de los dos lados tiene FK en su propia tabla**. JPA genera una **tabla intermedia** para representar la relación.
- Se usa `@ManyToMany` en ambos lados, y `@JoinTable` en el lado dueño para definir explícitamente esa tabla intermedia.

¿Qué tabla adicional genera JPA? ¿Qué columnas tiene? Definirla explícitamente usando `@JoinTable`.

- Son **dos relaciones independientes**, entonces se necesitan **dos tablas join distintas** (ver código abajo)

- En la db terminamos teniendo dos tablas intermedias nuevas cada una con los IDs de las tablas que relacionan formando una PK compuesta.

En la clase Route

```
@ManyToMany
@JoinTable(
    name = "route_drivers",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "driver_id")
)
private List<DriverUser> driverList;

@ManyToMany
@JoinTable(
    name = "route_tour_guides",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "tour_guide_id")
)
private List<TourGuideUser> tourGuideList;
```

En la clase DriverUser

```
@ManyToMany(mappedBy = "driverList")
private List<Route> routes = new ArrayList<>();
```

En la clase TourGuideUser

```
@ManyToMany(mappedBy = "tourGuideList")
private List<Route> routes = new ArrayList<>();
```

¿Pueden ambas relaciones compartir la misma tabla join? ¿Por qué?

- No, y hay dos razones para eso:
 - **Razón técnica:** DriverUser y TourGuideUser son tipos distintos. Una fila en la tabla join apunta a un usuario específico con un rol específico. Mezclarlos en una sola tabla haría imposible saber si el user_id de una fila es un chofer o un guía sin agregar una columna discriminadora extra, lo cual no es el modelo que JPA espera.
 - **Razón del dominio:** El enunciado dice explícitamente que un chofer y un guía turístico tienen **responsabilidades distintas** en el recorrido. Una misma persona podría ser chofer en un recorrido y guía en otro, si tenemos una sola tabla la información podría verse contradictoria.

18. La relación Purchase -> Review es opcional (0..1). Implementar el mapeo de ambos lados.
¿Cómo se representa la opcionalidad en JPA?

En la clase Review

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"review_seq")
@SequenceGenerator(name = "review_seq", sequenceName = "review_sequence",
allocationSize = 50)
private Long id;

@Column(nullable = false)
private int rating;

@Column
private String comment;

@OneToOne
@JoinColumn(name = "purchase_id", nullable = false)
private Purchase purchase;
```

En la clase Purchase

```
@OneToOne(mappedBy = "purchase", optional = true)
private Review review;
```

El parámetro `optional` le indica a **Hibernate** qué tipo de **JOIN** usar al hacer consultas:

- **optional = true** → LEFT JOIN (trae la Purchase aunque no tenga Review)
- **optional = false** → INNER JOIN (solo trae Purchases que tengan Review)

19. ItemService referencia a Service (muchos-a-uno). Analizar el diagrama: ¿es navegable esta relación desde Service hacia ItemService? Justificar si conviene hacerla bidireccional o no.

Si, la relación es navegable en los dos sentidos:

- `Service` tiene `-items *` apuntando hacia `ItemService` → **Service conoce sus items**
- `ItemService` tiene `-service 1` apuntando hacia `Service` → **ItemService conoce su Service**

En cuanto a hacerla bidireccional o no, hay 2 opciones:

- *Opción A — Mantenerla bidireccional pero con carga lazy:*

- Lazy haría que Hibernate no cargue la lista de items hasta que explícitamente se necesite, evitando que se cargue una lista muy grande cada vez que se cargue un servicio en memoria.
- *Opción B — Eliminar la colección de `Service` y hacerla unidireccional:*
 - La justificación de esto es que para este dominio, lo más probable es que la navegación relevante sea siempre desde `purchase` hacia sus `itemServiceList`, no desde `service`.

Yo dejo la **Opción A** porque no sé si podemos tomar esa decisión sobre el dominio que te plantean ellos.

20. Implementar el mapeo completo de las siguientes entidades con todas sus relaciones, siguiendo las anotaciones y decisiones discutidas: Supplier, Purchase, ItemService, Route, Stop y Review. Para cada relación bidireccional, incluir las anotaciones en ambos lados.

```
@Entity
@Table(name = "suppliers")
public class Supplier {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"supplier_seq")
    @SequenceGenerator(name = "supplier_seq", sequenceName =
"supplier_sequence", allocationSize = 50)
    private Long id;

    @Column(nullable = false)
    private String businessName;

    @Column(nullable = false, unique = true)
    private String authorizationNumber;

    @OneToMany(mappedBy = "supplier", fetch = FetchType.LAZY)
    private List<Service> services;

    //getters y setters
}
```

```
@Entity
@Table(name = "stops")
public class Stop {

    @Id
```

```

    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"stop_seq")
    @SequenceGenerator(name = "stop_seq", sequenceName = "stop_sequence",
allocationSize = 50)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column
    private String description;

    //getters y setters
}

```

```

@Entity
@Table(name = "routes")
public class Route {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"route_seq")
    @SequenceGenerator(name = "route_seq", sequenceName = "route_sequence",
allocationSize = 50)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private float price;

    @Column(nullable = false)
    private float totalKm;

    @Column(nullable = false)
    private int maxNumberUsers;

    @ManyToMany(fetch = FetchType.EAGER,
        cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    @JoinTable(
        name = "route_stops",
        joinColumns = @JoinColumn(name = "route_id"),
        inverseJoinColumns = @JoinColumn(name = "stop_id")
    )
}

```

```

private List<Stop> stops;

@ManyToMany
@JoinTable(
    name = "route_drivers",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "driver_id")
)
private List<DriverUser> driverList;

@ManyToMany
@JoinTable(
    name = "route_tour_guides",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "tour_guide_id")
)
private List<TourGuideUser> tourGuideList;

//getters y setters
}

```

```

@Entity
@Table(name = "item_services")
public class ItemService {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"item_service_seq")
    @SequenceGenerator(name = "item_service_seq", sequenceName =
"item_service_sequence", allocationSize = 50)
    Long id;

    @Column(nullable = false)
    private int quantity;

    @ManyToOne
    @JoinColumn(name = "purchase_id", nullable = false)
    private Purchase purchase;

    @ManyToOne
    @JoinColumn(name = "service_id", nullable = false)
    private Service service;

    //getters y setters
}

```

```

@Entity
@Table(name = "purchases")
public class Purchase {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"purchase_seq")
    @SequenceGenerator(name = "purchase_seq", sequenceName =
"purchase_sequence", allocationSize = 50)
    Long id;

    @Column(nullable = false, unique = true)
    private String code;

    @Column(nullable = false)
    private float totalPrice;

    @Column(nullable = false)
    private Date date;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne
    @JoinColumn(name = "route_id", nullable = false)
    private Route route;

    @OneToOne(mappedBy = "purchase", optional = true)
    private Review review;

    @OneToMany(mappedBy = "purchase", cascade = CascadeType.ALL,
orphanRemoval = true)
    private List<ItemService> itemServiceList;

    public void addItem(ItemService item) {
        itemServiceList.add(item);
        item.setPurchase(this);
    }

    public void removeItem(ItemService item) {
        itemServiceList.remove(item);
        item.setPurchase(null);
    }
}

```

```
//getters y setters  
}
```

```
@Entity  
@Table(name = "reviews")  
public class Review {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =  
"review_seq")  
    @SequenceGenerator(name = "review_seq", sequenceName =  
"review_sequence", allocationSize = 50)  
    private Long id;  
  
    @Column(nullable = false)  
    private int rating;  
  
    @Column  
    private String comment;  
  
    @OneToOne  
    @JoinColumn(name = "purchase_id", nullable = false)  
    private Purchase purchase;  
  
    //getters y setters  
}
```

2.3 Fetch Type: LAZY vs. EAGER

21. ¿Para qué sirve la propiedad fetch en las anotaciones de relación? ¿Cuáles son los valores posibles? ¿Cuál es el valor por defecto para @OneToMany, para @ManyToOne y para @ManyToMany?

La propiedad **fetch** determina la **estrategia de carga** de los objetos asociados en una relación. Define si los objetos relacionados deben traerse de la base de datos en el mismo momento en que se carga la entidad principal, o si deben recuperarse únicamente cuando se acceda a ellos por primera vez en el código. Esta decisión impacta directamente en la performance (número de consultas SQL) y en el espacio que ocupa el grafo de objetos en la memoria del servidor.

Valores Posibles

- `FetchType.LAZY`
 - Los datos relacionados **no se cargan inmediatamente**.
 - Hibernate utiliza el patrón **Proxy**, colocando un "sustituto" en lugar del objeto real.

- La carga física de los datos ocurre solo cuando se invoca un método (ej. un *getter*) sobre la relación.
- **Ventaja:** Ahorra memoria y evita traer grafos de objetos masivos que quizás no se utilicen
- `FetchType.EAGER`
 - Los datos relacionados **se cargan obligatoriamente** junto con la entidad principal, usualmente mediante un `JOIN` en la consulta SQL.
 - **Ventaja:** Útil cuando se sabe con certeza que los datos relacionados serán requeridos siempre, evitando múltiples consultas posteriores (*N+1 queries*).

Anotación	Valor por Defecto	Justificación
@OneToMany	LAZY	Al tratarse de colecciones que pueden ser muy grandes, el default razonable es no cargarlas para evitar agotar la memoria
@ManyToOne	EAGER	En el estándar JPA oficial es EAGER , sin embargo, Hibernate suele tratar estas relaciones también como LAZY por defecto para ser conservador con los recursos
@ManyToMany	LAZY	Similar a las colecciones 1..N, se busca evitar la hidratación accidental de grandes partes del grafo de objetos

22. Describir ventajas y desventajas concretas de EAGER y LAZY, en términos de performance de acceso como de espacio en memoria. ¿Por qué configurar EAGER en todas las relaciones suele ser una mala idea en aplicaciones reales?

`FetchType.EAGER` :

- **Ventajas:**
 - **Performance de acceso:** Los datos están disponibles de inmediato en memoria. Se evitan consultas adicionales posteriores (problema de las N+1 queries) si se sabe de antemano que la relación será utilizada siempre.
 - **Simplicidad transaccional:** No existe riesgo de errores al acceder a los datos fuera del contexto de una sesión activa.
- **Desventajas:**
 - **Espacio en memoria:** Se "hidratan" (cargan) objetos que quizás nunca se utilicen, ocupando espacio innecesario en la memoria del servidor.
 - **Performance inicial:** La consulta inicial es mucho más pesada y lenta debido al procesamiento de múltiples `JOINS` complejos en el motor de base de datos.

`FetchType.LAZY`

- **Ventajas:**
 - **Eficiencia de memoria:** Solo se carga lo estrictamente necesario. Evita traer grafos de objetos masivos a la memoria, lo cual es vital para la escalabilidad.
 - **Performance de arranque:** La recuperación de la entidad inicial es extremadamente rápida.
- **Desventajas:**
 - **Riesgo de LazyInitializationException :** Si se intenta acceder a una relación LAZY cuando la Session ya ha sido cerrada, la aplicación fallará.
 - **Performance de acceso diferido:** Si se requieren muchos objetos relacionados en un bucle, se pueden disparar cientos de consultas individuales a la base de datos, degradando el rendimiento.

No se debe configurar **EAGER** en todas las relaciones por los siguientes motivos:

- **Efecto Cascada de Hidratación (Chain Reaction):** En un sistema orientado a objetos, los grafos no tienen límites claros. Si todas las relaciones son EAGER, cargar un solo objeto (ej. un User) podría desencadenar la carga de sus Purchases , lo que a su vez cargaría todos los ItemServices , sus Services , los Suppliers de esos servicios, y así sucesivamente.
- **Agotamiento de Memoria:** Esta reacción en cadena puede terminar intentando cargar gran parte de la base de datos completa en la memoria de un solo nodo de aplicación al realizar una consulta simple, lo que provoca errores de OutOfMemory.
- **Explosión de JOINS:** A nivel de SQL, el ORM intentará generar consultas con una cantidad ingente de JOINS . Las bases de datos relacionales pierden eficiencia drásticamente a medida que aumenta el número de tablas vinculadas en una sola sentencia.
- **Pérdida de Control:** El desarrollador pierde la capacidad de optimizar casos de uso específicos. Al ser EAGER una configuración estática en la entidad, no se puede "apagar" para consultas donde solo se necesita un dato básico, obligando a la aplicación a pagar siempre el costo máximo de carga.

23. Para cada relación del modelo, elegir el FetchType más adecuado y justificar. Luego implemente su decisión en el proyecto tours.

Relación	Tipo de anotación	FetchType elegido	Justificación
Purchase -> User	@ManyToOne	LAZY	Al cargar una compra no siempre se necesitan todos los datos del usuario
Purchase -> Route	@ManyToOne	EAGER	Al mostrar una compra siempre se necesita saber qué recorrido se compró

Relación	Tipo de anotación	FetchType elegido	Justificación
Purchase -> itemServiceList	@OneToMany	EAGER	El enunciado dice explícitamente que el usuario debe poder ver el detalle de lo abonado
Purchase -> Review	@OneToOne	LAZY	La review es opcional (0 .. 1) y no siempre se necesita al cargar la compra
ItemService -> Service	@ManyToOne	EAGER	Al mostrar un ítem siempre se necesita saber qué servicio es (nombre, precio)
Route -> stops	@OneToMany → @ManyToMany por enunciado	LAZY	Se podría llegar a requerir más visualizar una ruta por separado que con sus paradas
Route -> drivers (DriverUser)	@ManyToMany	LAZY	Los choferes solo se necesitan en el detalle operativo, no en toda consulta de ruta
Route -> tourGuides (TourGuideUser)	@ManyToMany	LAZY	Igual que drivers
User -> purchases	@OneToMany	LAZY	Un usuario puede tener muchas compras, cargarlas siempre sería costoso
Service -> supplier	@ManyToOne	LAZY	Al consultar un servicio raramente se necesita el proveedor inmediatamente

24. ¿Cómo podría producirse una `LazyInitializationException` en el modelo? Investigue de qué representa esta excepción y escribir un escenario concreto explicando al menos formas de resolverlo sin cambiar el `FetchType` a `EAGER`.

Una `LazyInitializationException` es una excepción que Hibernate lanza cuando intentamos acceder a una colección o relación marcada como `LAZY` después de que la sesión de Hibernate ya fue cerrada. Hibernate necesita la sesión abierta para poder ir a buscar los datos a la base de datos bajo demanda, si la sesión ya no existe, no puede hacerlo y explota.

Ejemplo que podría pasar en nuestro modelo

```
// En algún servicio o repositorio:
public Purchase getPurchase(Long id) {
```

```

    Session session = sessionFactory.openSession();
    Purchase purchase = session.get(Purchase.class, id);
    session.close(); // ← sesión cerrada acá
    return purchase;
}

// En otro lado del código (un controller, por ejemplo):
Purchase purchase = getPurchase(1L);
User user = purchase.getUser(); // ← LazyInitializationException

```

Formas de resolverlo sin cambiar a EAGER

- **JOIN FETCH** *en la query*
 - Indicarle explícitamente a Hibernate en la consulta que traiga la relación junto con la entidad principal, usando HQL (ver código abajo). Esta es la mejor forma casi siempre cuando se sabe cuál relación LAZY necesito fuera de la sesión.

```

public Purchase getPurchaseWithUser(Long id) {
    Session session = sessionFactory.openSession();

    Purchase purchase = session.createQuery(
        "SELECT p FROM Purchase p JOIN FETCH p.user WHERE p.id = :id",
        Purchase.class
    )
    .setParameter("id", id)
    .uniqueResult();

    session.close();
    return purchase; // user ya está cargado, no necesita sesión
}

```

- *Mantener la sesión abierta con* **@Transactional**
 - Anotar el método con **@Transactional** para que Spring mantenga la sesión abierta durante toda la ejecución del método (ver código abajo). Sirve cuando se necesita navegar el grafo.

```

@Transactional
public void mostrarUsuarioDeLaCompra(Long id) {
    Purchase purchase = purchaseRepository.findById(id);
    User user = purchase.getUser();
    System.out.println(user.getName());
} // ← la sesión se cierra recién acá, al terminar el método

```

2.4 Operaciones en cascada

25. Enumerar todos los valores de CascadeType y explicar qué operación propaga cada uno sobre las entidades relacionadas.

Valores de CascadeType y Operaciones que Propagan

- `CascadeType.PERSIST` : Propaga la operación de creación. Cuando se persiste la entidad padre (vía `persist()`), todas las entidades relacionadas que sean transitorias (*transient*) también se persistirán automáticamente. Es la base técnica para que funcione la persistencia por alcance.
 - `CascadeType.MERGE` : Propaga la operación de unión. Si una entidad desasociada (*detached*) se reincorpora al contexto de persistencia mediante `merge()`, los cambios en sus objetos relacionados también se fusionarán con el estado de la base de datos.
 - `CascadeType.REMOVE` : Propaga la eliminación. Al eliminar un objeto padre (ej. vía `delete()`), Hibernate eliminará físicamente de la base de datos todos los registros asociados en esa relación. Es común utilizarlo en relaciones de *composición*, como entre `Route` y `Stop` o `Purchase` e `ItemService`.
 - `CascadeType.REFRESH` : Propaga la recarga de datos. Si se invoca `refresh()` sobre una entidad para sincronizar su estado con los valores actuales de la base de datos, las entidades relacionadas también volverán a cargarse desde el disco.
 - `CascadeType.DETACH` : Propaga la desasociación. Si el objeto padre es expulsado del *Persistence Context* (vía `detach()` o `evict()`), todos sus conocidos vinculados en la relación pasarán también al estado *detached*.
 - `CascadeType.ALL` : Es un valor de conveniencia que *engloba todas las operaciones anteriores* (`persist`, `merge`, `remove`, `refresh` y `detach`). Se utiliza frecuentemente cuando el ciclo de vida de los objetos relacionados depende totalmente del padre.
26. ¿Cuál es el comportamiento por defecto cuando no se define cascade? ¿Cuál es la finalidad general de CASCADE? Proponga un caso del modelo donde definir un CASCADE inadecuado podría traer problemas a la consistencia de la base de datos.

En el estándar *JPA*, el comportamiento por defecto cuando no se define explícitamente la propiedad `cascade` es que *no se propaga ninguna operación*. Esto significa que cualquier acción realizada sobre una entidad (como guardarla o eliminarla) afectará únicamente a ese objeto individual; los objetos relacionados en el grafo no sufrirán ningún cambio automático en su estado de persistencia.

La *finalidad* primordial de las operaciones en cascada es automatizar la gestión del ciclo de vida de los objetos relacionados, permitiendo implementar el principio de *persistencia por alcance*.

Caso de Cascade inadecuado: `CascadeType.REMOVE` en `Service` → `ItemService`

- Si definimos en la Clase **Service** esto:

```
@OneToMany(mappedBy = "service", cascade = CascadeType.REMOVE) private
List<ItemService> itemServiceList;
```

- El **problema** ocurre de esta forma por ejem.:
 - El proveedor "Artesanías Boca" decide dejar de ofrecer el servicio "Foto en el Obelisco".
 - El administrador borra el Service de la base de datos: `session.remove(service)`.
 - `CascadeType.REMOVE` propaga el borrado hacia todos sus `ItemService`.
 - Esos `ItemService` pertenecen a `Purchases` ya confirmadas y pagadas, por lo tanto, ahora las compras quedan **incompletas e inconsistentes**: el usuario pagó por un servicio que ya no aparece en su compra. El historial de lo abonado queda corrupto, justamente lo que el enunciado dice que debe preservarse: *"para que el usuario pueda tener control de lo abonado"*

27. ¿Cuál es la diferencia entre `cascade = REMOVE` y `orphanRemoval = true`? ¿Pueden usarse conjuntamente? Ejemplificar con el par `Purchase -> ItemService`.

`CascadeType.REMOVE`

- **Definición:** Propaga la operación de eliminación (`delete` o `remove`) desde la entidad padre hacia sus hijos.
- **Disparador:** Solo se activa cuando se invoca explícitamente `session.delete(padre)`.
- **Comportamiento:** Si eliminas una compra, Hibernate buscará todos los ítems asociados y los borrará de la base de datos.
- **Limitación:** Si simplemente quitas un ítem de la lista (desasociación), Hibernate no lo borrará de la base de datos; el objeto quedará "huérfano" en el disco.

`orphanRemove = true`

- **Definición:** Es un mecanismo más agresivo diseñado específicamente para relaciones de **composición**.
- **Disparador:** Se activa tanto por la eliminación del padre como por la **desreferenciación** de un hijo dentro de una colección.
- **Comportamiento:** Si remueves un objeto de la colección (ej. `compra.getItems().remove(item)`), Hibernate detecta que ese objeto ya no tiene un padre que lo contenga y lo elimina físicamente de la base de datos al realizar el **flush**.
- **Alcance:** Cubre todo lo que hace `cascade = REMOVE` y añade la limpieza de objetos huérfanos por pérdida de referencia.

Sí, pueden usarse conjuntamente, aunque en la práctica `orphanRemoval = true` hace redundante la necesidad de `REMOVE`.

28. Para la relación Purchase -> ItemService (composición):

1. ¿Qué tipos de cascade configurarías? Justificar cada uno.
2. ¿Usarías `orphanRemoval`? ¿Por qué?
3. Describir qué ocurre a nivel de base de datos cuando se elimina un `ItemService` de la lista de una `Purchase` y Hibernate realiza el flush.

¿Qué tipos de cascade configurarías? Justificar cada uno.

- `PERSIST`: Sin usar el `PERSIST` se tendría que hacer `session.persist(item)` manualmente por cada ítem.

```
Purchase purchase = new Purchase();
ItemService item = new ItemService();
purchase.addItemService(item);
session.persist(purchase); // persiste también el ItemService automáticamente
```

- `MERGE`: Sin esto, los cambios en los ítems no se guardarían al mergear la compra.

```
// Si una Purchase viene de fuera de la sesión (detached) con ítems modificados
session.merge(purchase); // propaga el merge a los ItemService modificados
```

- `REMOVE`: Sin esto, borrar una compra lanzaría una excepción de integridad referencial porque los ítems tienen FK hacia la compra.

```
session.remove(purchase); // elimina también todos sus ItemService
```

- `REFRESH`: Es discutible, no es estrictamente necesario. Refrescar una compra recargando sus ítems desde la BD tiene sentido solo en escenarios con alta concurrencia donde otro proceso pudo haber modificado los ítems.
- `DETACH`: No sería necesario, no hay un caso de uso del dominio donde se necesite desvincular una compra y que sus ítems también se desvinculen.

```
@OneToMany(mappedBy = "purchase",
            cascade = {
                CascadeType.PERSIST,
                CascadeType.MERGE,
            })
```

```

        CascadeType.REMOVE
    },
    orphanRemoval = true,
    fetch = FetchType.EAGER)
private List<ItemService> itemServiceList = new ArrayList<>();

```

¿Usarias orphanRemoval? ¿Por qué?

- **Sí**, un `ItemService` es parte de la composición de `Purchase`. No tiene existencia propia fuera de ella. Si el usuario decide quitar un servicio de su compra antes de confirmarla por ejem. sin el `orphanRemoval`, ese `ItemService` quedaría huérfano en la tabla `item_services` con un `purchase_id` que ya no referencia nada lógicamente válido, corrompiendo la base de datos silenciosamente.

Describir qué ocurre a nivel de base de datos cuando se elimina un `ItemService` de la lista de una `Purchase` y Hibernate realiza el flush.

- El flujo sería esto:

```

// 1. Cargás la compra (sesión abierta)
Purchase purchase = session.get(Purchase.class, 1L);

// 2. Quitás un ítem de la colección
ItemService item = purchase.getItemServiceList().get(0);
purchase.getItemServiceList().remove(item);

// 3. Hibernate hace flush (al cerrar transacción o manualmente)
session.flush();

```

- Lo que pasa internamente es lo siguiente:
 - Al cargar la **Purchase**, **Hibernate** toma una "foto" del estado original de `itemServiceList` en el Persistence Context.
 - Al hacer `.remove(item)`, la lista en memoria cambia pero la BD todavía no sabe nada.
 - En el **flush**, Hibernate compara el estado actual con la "foto" original y detecta que falta un ítem.
 - Como `orphanRemoval = true`, **Hibernate** genera: `DELETE FROM item_services WHERE id = ?`

29. Para la relación Purchase -> Review:

1. ¿Qué cascades configurarías?
2. Si se elimina una Purchase, ¿debería eliminarse también su Review? Justificar desde el modelo de negocio.

¿Qué cascades configurarías?

- **PERSIST** : Sin esto se tendría que persistir la **Review** manualmente cada vez que se crea junto con la compra.

```
Purchase purchase = new Purchase();
Review review = new Review();
review.setPurchase(purchase);
purchase.setReview(review);
session.persist(purchase); // persiste también la Review automáticamente
```

- **MERGE** : Es necesario, si una **Purchase** viene detached con su **Review** modificada, el merge debe propagarse para que los cambios en el comentario o el rating se guarden.
- **REMOVE** : Es necesario, si se elimina una **Purchase**, su **Review** debe eliminarse también. Sin este cascade quedaría una **Review** huérfana en la BD con una FK hacia una compra que ya no existe.
- **REFRESH** y **DETACH** : No serían necesarios por el mismo razonamiento del punto anterior.

```
@OneToOne(mappedBy = "purchase",
    optional = true,
    fetch = FetchType.LAZY,
    cascade = {
        CascadeType.PERSIST,
        CascadeType.MERGE,
        CascadeType.REMOVE
    })
private Review review;
```

Si se elimina una **Purchase**, ¿debería eliminarse también su **Review**? Justificar desde el modelo de negocio.

- Si, debería de eliminarse también la **Review**, una **Review** **no tiene identidad propia** fuera de su compra. No existe "una reseña" en abstracto — existe "la reseña de esta compra puntual de este usuario sobre este recorrido". Si la compra desaparece, la reseña pierde todo su contexto y significado.

30. Para la relación **Supplier** -> **Service**:

1. ¿Qué cascades tienen sentido?
2. Si se elimina un **Supplier**, ¿qué debería ocurrir con sus **Service**? ¿Y con las **Purchase** que los contienen a través de **ItemService**?

¿Qué cascades tienen sentido?

- **PERSIST** : Es razonable que al dar de alta un proveedor nuevo con sus servicios, todo se persista junto.

```
Supplier supplier = new Supplier();
Service service = new Service();
service.setSupplier(supplier);
supplier.getServices().add(service);
session.persist(supplier); // persiste también los Service automáticamente
```

- **MERGE** : Tiene sentido, si un **Supplier** viene detached con servicios modificados, el merge debería propagarse para actualizar los datos de los servicios asociados.
- **REMOVE** : No tiene sentido, este es el caso más importante y se justifica en el segundo inciso del punto.
- **REFRESH** y **DETACH** : No necesarios por razones parecidas a las de los puntos anteriores.

```
@OneToMany(mappedBy = "supplier",
            fetch = FetchType.LAZY,
            cascade = {
                CascadeType.PERSIST,
                CascadeType.MERGE
            })
private List<Service> services = new ArrayList<>();
```

Si se elimina un **Supplier**, ¿qué debería ocurrir con sus **Service**? ¿Y con las **Purchase** que los contienen a través de **ItemService**?

- Si se hubiera puesto **CascadeType.REMOVE** cuando se borrara un **Supplier** el cascade propaga ese delete a los **Service** lo que conlleva a 2 casos posibles:
 - **Sin cascade en Service → ItemService**: la BD lanza error porque **item_services** apunta a un **service** que ya no existe.
 - **Con cascade en Service → ItemService**: se borran silenciosamente ítems de **compras ya pagadas**, dejando el historial corrupto.
 - Por esta razón la mejor estrategia sería realizar una **baja lógica** del **Supplier** para que en vez de que se borre físicamente, se marque como inactivo, de esta forma todo el historial de compras queda intacto. El proveedor simplemente deja de aparecer en el sistema para nuevas operaciones, pero su huella en las compras pasadas se preserva.
31. ¿Por que **CascadeType.REMOVE** en una relación **@ManyToMany** (por ejemplo **Route <-> DriverUser**) suele ser peligroso? Describir un escenario donde su uso cause pérdida no deseada de datos.

En una relación @ManyToMany ambos lados son entidades **independientes** que simplemente se asocian entre sí. Ninguna "posee" a la otra. Un DriverUser existe por sí mismo y puede estar en múltiples rutas, y una Route existe independientemente de sus choferes.

El problema es que CascadeType.REMOVE **no entiende de multiplicidad**: cuando se borra una entidad, propaga el DELETE hacia todas las entidades relacionadas sin considerar si esas entidades tienen otras relaciones vigentes.

Esto quiere decir que podemos tener un caso donde se pierda datos de manera indeseada si por ejemplo borramos una Route que tenga asociada un DriverUser ya que se propagan los delete, de esta forma **al solo borrar una ruta, perdemos también al chofer**.

32. Implemente las operaciones en cascada adecuada para todas las relaciones del modelo tours.

2.5 Jerarquía de herencia: User, DriverUser y TourGuideUser

33. Describir las tres estrategias de mapeo de herencia. Para cada una indicar qué tablas se crean, si aparece columna discriminadora y cómo resuelve Hibernate una consulta polimórfica. Completar la tabla:

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Tablas creadas en la BD	Una sola tabla para toda la jerarquía	Una para la superclase y una por cada subclase	Una tabla por cada subclase concreta
Columna discriminadora	Sí (obligatoria para identificar el tipo de objeto)	Opcional (Hibernate puede deducirlo por el JOIN)	No (el tipo está implícito en la tabla)
NULLs en columnas de subclases	Sí (obligatoria para identificar el tipo de objeto)	No (cada tabla tiene solo sus datos específicos)	No (las tablas son independientes y completas)
Consulta polimórfica "todos los Users"	Muy eficiente: Simple SELECT sobre una tabla	Ineficiente: Requiere múltiples JOINS con todas las hijas	Muy ineficiente: Requiere un UNION ALL de todas las tablas
Cargar un DriverUser por ID	Eficiente: SELECT con filtro por ID y discriminador	Regular: Requiere un JOIN entre la tabla padre e hija	Eficiente: SELECT directo a la tabla DRIVER_USER
Integridad referencial (FK posibles)	Limitada: Atributos de subclases no	Alta : Se pueden definir	Alta : Cada tabla es independiente y soporta restricciones nativas

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
	pueden ser NOT NULL en DB	restricciones NOT NULL y FKs reales	
Performance en lecturas simples (de una entidad)	Excelente (sin joins)	Baja (siempre requiere joins para completar el objeto)	Buena (lectura directa de la tabla concreta)
Que implica para agregar nueva subclase	Agregar nuevas columnas a la tabla existente	Crear una nueva tabla para la subclase	Compleja: Estructura repetida y afecta a las consultas UNION

Estrategias de Mapeo de Herencia

- **SINGLE_TABLE**
 - Es la estrategia por defecto en JPA. Consiste en mapear todas las clases de la jerarquía a una **única tabla** que contiene la unión de todos los atributos de todas las subclases.
 - **Consulta polimórfica:** Hibernate realiza un simple `SELECT *` sobre la tabla.
 - **Columna discriminadora:** Es obligatoria para que Hibernate sepa a qué clase concreta pertenece cada fila.
 - **JOINED**
 - Esta estrategia sigue las reglas de normalización de bases de datos. Se crea una tabla para la superclase y una tabla adicional por cada subclase que contenga solo sus atributos específicos. Las tablas se vinculan mediante la clave primaria (PK) que actúa también como clave foránea (FK) hacia la tabla padre.
 - **Consulta polimórfica:** Hibernate debe realizar `OUTER JOINS` entre la tabla padre y todas las tablas hijas para reconstruir el grafo de objetos.
 - **TABLE_PER_CLASS**
 - También conocida como "estrategia de unión", mapea cada clase concreta a su propia tabla. Los atributos de la superclase se duplican en cada una de las tablas de las subclases.
 - **Consulta polimórfica:** Para obtener "todos los usuarios", Hibernate debe ejecutar una sentencia `UNION` entre todas las tablas involucradas, lo cual es una operación costosa en SQL.
34. Implementar el mapeo de la jerarquía User/DriverUser/TourGuideUser usando la estrategia SINGLE_TABLE. Especificar @Inheritance, @DiscriminatorColumn y @DiscriminatorValue para cada clase. Incluir todos los atributos.
1. Ejecutar los tests provistos y analizar el DDL generado: ¿cuántas tablas aparecen? ¿Qué columnas tiene la tabla? ¿Dónde están los atributos expedient y education?

2. ¿Qué ocurre con las columnas de subclase cuando se inserta un TourGuideUser? ¿Y cuando se inserta un DriverUser?
3. ¿Cuáles son las ventajas y desventajas de esta estrategia para este modelo concreto?

```
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "user_type", discriminatorType =
DiscriminatorType.STRING)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String name;

    @Column
    private String email;

    @Column
    private Date birthdate;

    @Column
    private String phoneNumber;

    @Column(nullable = false)
    private boolean active;

    @OneToMany(
        mappedBy = "user",
        fetch = FetchType.LAZY,
        cascade = {
            CascadeType.PERSIST,
            CascadeType.MERGE
```

```

    })
    private List<Purchase> purchaseList;

    //getter y setters existentes
}

```

```

@Entity
@DiscriminatorValue("DRIVER")
public class DriverUser extends User {

    @Column
    private String expedient;

    @ManyToMany(mappedBy = "driverList")
    private List<Route> routes;

    //getters y setters existentes
}

```

```

@Entity
@DiscriminatorValue("TOUR_GUIDE")
public class TourGuideUser extends User {

    @Column
    private String education;

    @ManyToMany(mappedBy = "tourGuideList")
    private List<Route> routes;

    //getters y setters existentes
}

```

Ejecutar los tests provistos y analizar el DDL generado: ¿cuántas tablas aparecen? ¿Qué columnas tiene la tabla? ¿Dónde están los atributos expedient y education?

- Con `SINGLE_TABLE` se crea **una sola tabla** llamada `users` que contiene **todas las columnas de las tres clases juntas**, `expedient` y `education` están **en la misma tabla** `users`, como columnas nullable, porque pertenecen a subclases y no siempre tendrán valor:

```

CREATE TABLE users (
    -- columna discriminadora
    user_type    VARCHAR(31) NOT NULL,  ∈ "USER", "DRIVER" o "TOUR_GUIDE"

```

```

-- columnas de User (clase base)
id            BIGINT      NOT NULL,
username      VARCHAR(255) NOT NULL UNIQUE,
password      VARCHAR(255) NOT NULL,
name          VARCHAR(255) NOT NULL,
email         VARCHAR(255),
birthdate     DATETIME,
phoneNumber   VARCHAR(255),
active        BOOLEAN     NOT NULL,

-- columna de DriverUser
expedient     VARCHAR(255),          ← nullable porque no aplica a User
ni TourGuideUser

-- columna de TourGuideUser
education     VARCHAR(255),          ← nullable porque no aplica a User
ni DriverUser

PRIMARY KEY (id)
)

```

¿Qué ocurre con las columnas de subclase cuando se inserta un TourGuideUser? ¿Y cuando se inserta un DriverUser?

- Al insertar un TourGuideUser lo que hace Hibernate es:

```

INSERT INTO users (user_type, id, username, password, name, email,
birthdate, phoneNumber, active, education, expedient)
VALUES ('TOUR_GUIDE', 1, 'userG', '1234', 'Usuario TourGuide', ...,
'edu...', NULL)
      ↑           ↑
education llena expedient en NULL

```

- Al insertar un DriverUser lo que hace Hibernate es:

```

INSERT INTO users (user_type, id, username, password, name, email,
birthdate, phoneNumber, active, education, expedient)
VALUES ('DRIVER', 2, 'userD', '1234', 'Usuario Driver', ...,
NULL, 'exp...')
      ↑           ↑
education en NULL expedient lleno

```

¿Cuáles son las ventajas y desventajas de esta estrategia para este modelo concreto?

- **Ventajas:**
 - **Consultas polimórficas simples:** el test `getUserById()` devuelve cualquier tipo de `User` con un único `SELECT` sin joins, porque todo está en una sola tabla.
 - **Performance en lectura:** cargar un `DriverUser` por ID es un `SELECT` directo sobre `users` filtrando por `id`, sin necesidad de hacer join con otra tabla.
 - **Simplicidad del esquema:** una sola tabla para gestionar, fácil de entender y mantener para un emprendimiento pequeño como este.
- **Desventajas:**
 - **Columnas nullable obligatorias:** `expedient` y `education` no pueden tener `NOT NULL` en la BD porque cada una solo aplica a una subclase. Esto debilita la integridad referencial — la base de datos no puede garantizar que un `DriverUser` siempre tenga `expedient`.
 - **Tabla que crece horizontalmente:** si en el futuro se agregan más subclases de `User` (por ejemplo `AdminUser` con sus propios atributos), la tabla `users` acumula más columnas nullable, volviéndose más difícil de mantener.
 - **Desperdicio de espacio:** cada fila de un `User` base tiene `expedient = NULL` y `education = NULL`. Cada fila de `DriverUser` tiene `education = NULL`. Con muchos usuarios esto representa espacio desperdiciado.
 - **Sin integridad a nivel BD para atributos de subclase:** no hay forma de crear una constraint que diga "si `user_type = 'DRIVER'` entonces `expedient` no puede ser null" usando solo SQL estándar.

35. Reimplementar el mapeo de la misma jerarquía usando la estrategia JOINED.

1. Ejecutar los tests y analizar el DDL: ¿cuantas tablas aparecen? ¿Qué FK existe entre ellas?
2. Comparar el SQL generado por Hibernate al cargar un `DriverUser` en JOINED vs. en SINGLE_TABLE. ¿Cómo difiere?
3. ¿Cuáles son las ventajas y desventajas de JOINED para este modelo?

```
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.JOINED)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
allocationSize = 50)
    private Long id;
```

```

@Column(nullable = false, unique = true)
private String username;

@Column(nullable = false)
private String password;

@Column(nullable = false)
private String name;

@Column
private String email;

@Column
private Date birthdate;

@Column
private String phoneNumber;

@Column(nullable = false)
private boolean active;

@OneToMany(
    mappedBy = "user",
    fetch = FetchType.LAZY,
    cascade = {
        CascadeType.PERSIST,
        CascadeType.MERGE
    })
private List<Purchase> purchaseList;

//getters y setters existentes...
}

```

```

@Entity
public class DriverUser extends User {

    @Column
    private String expedient;

    @ManyToMany(mappedBy = "driverList")
    private List<Route> routes;

    //getters y setters existentes...
}

```

```

@Entity
public class TourGuideUser extends User {

    @Column
    private String education;

    @ManyToMany(mappedBy = "tourGuideList")
    private List<Route> routes;

    //getters y setters existentes...
}

```

Ejecutar los tests y analizar el DDL: ¿cuántas tablas aparecen? ¿Qué FK existe entre ellas?

- Con JOINED se crean tres tablas, una por clase. La FK en cada tabla de subclase apunta al `id` de `users`. Es decir, `driver_users.id = users.id` para el mismo registro. **No hay columna discriminadora** por defecto, aunque Hibernate puede agregarla opcionalmente para optimizar:

```

CREATE TABLE users (
    id          BIGINT      NOT NULL,
    username    VARCHAR(255) NOT NULL UNIQUE,
    password    VARCHAR(255) NOT NULL,
    name        VARCHAR(255) NOT NULL,
    email       VARCHAR(255),
    birthdate   DATETIME,
    phoneNumber VARCHAR(255),
    active      BOOLEAN     NOT NULL,
    PRIMARY KEY (id)
)

CREATE TABLE driver_users (
    id          BIGINT NOT NULL,
    expedient   VARCHAR(255),
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES users(id) ← FK hacia users
)

CREATE TABLE tour_guide_users (
    id          BIGINT NOT NULL,
    education   VARCHAR(255),
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES users(id) ← FK hacia users
)

```

Comparar el SQL generado por Hibernate al cargar un DriverUser en JOINED vs. en SINGLE_TABLE. ¿Cómo difiere?

- Con **SINGLE_TABLE** era un único SELECT sobre una sola tabla:

```
-- Simple, todo en una tabla
SELECT u.*
FROM users u
WHERE u.id = ?
-- Hibernate sabe el tipo por la columna user_type
```

- Con **JOINED** Hibernate necesita hacer un JOIN entre las tablas:

```
-- Necesita unir users con driver_users
SELECT u.id, u.username, u.password, u.name, u.email,
       u.birthdate, u.phoneNumber, u.active,
       d.expedient
FROM users u
INNER JOIN driver_users d ON u.id = d.id
WHERE u.id = ?
```

- Y una **consulta polimórfica** como `getUserByUsername()` que puede devolver cualquier tipo de User es más costosa todavía:

```
-- Hibernate no sabe de antemano si es User, Driver o TourGuide
SELECT u.id, u.username, ...,
       d.expedient,          ← puede ser NULL si no es DriverUser
       t.education           ← puede ser NULL si no es TourGuideUser
FROM users u
LEFT JOIN driver_users d    ON u.id = d.id
LEFT JOIN tour_guide_users t ON u.id = t.id
WHERE u.username = ?
```

¿Cuáles son las ventajas y desventajas de JOINED para este modelo?

- **Ventajas:**
 - **Integridad referencial real:** `expedient` puede ser `NOT NULL` en `driver_users` porque esa tabla solo tiene filas de choferes. La base de datos puede garantizar que todo `DriverUser` tenga `expedient`, cosa imposible en `SINGLE_TABLE`.
 - **Sin columnas nullable innecesarias:** la tabla `users` solo tiene los atributos base, sin columnas vacías por subclase.

- **Esquema más limpio:** refleja exactamente la jerarquía del dominio en la base de datos, es más fácil de entender para alguien que mira el esquema directamente.
- **Escala bien con nuevas subclases:** agregar un `AdminUser` solo requiere una nueva tabla sin tocar `users`.
- **Desventajas:**
 - **JOINS en cada consulta:** cargar cualquier subclase requiere al menos un JOIN. Para consultas polimórficas son LEFT JOINS en todas las tablas de subclase simultáneamente, lo cual es más costoso que SINGLE_TABLE.
 - **Performance inferior en lecturas simples:** el test `createAndGetUserTest()` que carga usuarios frecuentemente va a generar más tráfico hacia la BD que con SINGLE_TABLE.
 - **Impacto en este dominio concreto:** como `DriverUser` y `TourGuideUser` tienen **un solo atributo extra cada uno** (`expedient` y `education`), el beneficio de tener tablas separadas es mínimo comparado con el costo de los JOINS permanentes. Para un modelo tan simple, SINGLE_TABLE sería más eficiente en la práctica.

36. Realice el mismo proceso ahora con la estrategia TABLE_PER_CLASS. Indique cuál le parece la mejor estrategia para este modelo concreto y justifique su elección.

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String name;

    @Column
    private String email;

    @Column
    private Date birthdate;
```

```

@Column
private String phoneNumber;

@Column(nullable = false)
private boolean active;

@OneToMany(
    mappedBy = "user",
    fetch = FetchType.LAZY,
    cascade = {
        CascadeType.PERSIST,
        CascadeType.MERGE
    })
private List<Purchase> purchaseList;

//getters y setters existentes...
}

```

```

@Entity
@Table(name = "drivers_users")
public class DriverUser extends User {

    @Column
    private String expedient;

    @ManyToMany(mappedBy = "driverList")
    private List<Route> routes;

    //getters y setters existentes...
}

```

```

@Entity
@Table(name = "tour_guide_users")
public class TourGuideUser extends User {

    @Column
    private String education;

    @ManyToMany(mappedBy = "tourGuideList")
    private List<Route> routes;

    //getters y setters existentes...
}

```

DDL Generado ¿Cuántas tablas y qué columnas?

- Con TABLE_PER_CLASS se crean **tres tablas completamente independientes**, cada una con **todos los atributos** de la jerarquía repetidos. **No hay FKs entre tablas** — cada tabla es autónoma. Esto es la diferencia fundamental con JOINED:

```
CREATE TABLE users (  
    id            BIGINT            NOT NULL,  
    username      VARCHAR(255) NOT NULL,  
    password      VARCHAR(255) NOT NULL,  
    name          VARCHAR(255) NOT NULL,  
    email         VARCHAR(255),  
    birthdate     DATETIME,  
    phoneNumber   VARCHAR(255),  
    active        BOOLEAN          NOT NULL,  
    PRIMARY KEY (id)  
)  
  
CREATE TABLE driver_users (  
    id            BIGINT            NOT NULL,  
    username      VARCHAR(255) NOT NULL, ← repetido de User  
    password      VARCHAR(255) NOT NULL, ← repetido de User  
    name          VARCHAR(255) NOT NULL, ← repetido de User  
    email         VARCHAR(255),       ← repetido de User  
    birthdate     DATETIME,           ← repetido de User  
    phoneNumber   VARCHAR(255),       ← repetido de User  
    active        BOOLEAN          NOT NULL, ← repetido de User  
    expedient     VARCHAR(255),       ← propio de DriverUser  
    PRIMARY KEY (id)  
    -- sin FK hacia users ← tablas completamente independientes  
)  
  
CREATE TABLE tour_guide_users (  
    id            BIGINT            NOT NULL,  
    username      VARCHAR(255) NOT NULL, ← repetido de User  
    password      VARCHAR(255) NOT NULL, ← repetido de User  
    name          VARCHAR(255) NOT NULL, ← repetido de User  
    email         VARCHAR(255),       ← repetido de User  
    birthdate     DATETIME,           ← repetido de User  
    phoneNumber   VARCHAR(255),       ← repetido de User  
    active        BOOLEAN          NOT NULL, ← repetido de User  
    education     VARCHAR(255),       ← propio de TourGuideUser  
    PRIMARY KEY (id)  
    -- sin FK hacia users ← tablas completamente independientes  
)
```

SQL al cargar un DriverUser

- **Cargar por ID conocido** es simple. Sin JOINS, sin tabla padre. En este caso es igual de eficiente que SINGLE_TABLE.:

```
-- Hibernate sabe que es DriverUser, va directo a su tabla
SELECT d.id, d.username, d.password, d.name, d.email,
       d.birthdate, d.phoneNumber, d.active, d.expedient
FROM driver_users d
WHERE d.id = ?
```

- **Pero una consulta polimórfica** como `getUserByUsername()` es el gran problema, Hibernate genera un `UNION ALL` sobre las tres tablas. Es la consulta más costosa de las tres estrategias para este caso:

```
-- Hibernate no sabe en qué tabla está el usuario
-- debe buscar en las TRES tablas con UNION ALL
SELECT id, username, password, name, email, birthdate,
       phoneNumber, active, NULL as expedient, NULL as education
FROM users
UNION ALL
SELECT id, username, password, name, email, birthdate,
       phoneNumber, active, expedient, NULL as education
FROM driver_users
UNION ALL
SELECT id, username, password, name, email, birthdate,
       phoneNumber, active, NULL as expedient, education
FROM tour_guide_users
WHERE username = ?
```

¿Cuál es la mejor estrategia para este modelo?

- **La mejor estrategia es SINGLE_TABLE**, y la justificación viene directamente del dominio:
 - *Razón 1 — Las subclases tienen muy pocos atributos propios*
 - `DriverUser` agrega solo `expedient` y `TourGuideUser` agrega solo `education`. El costo de tener dos columnas nullable en `users` es mínimo comparado con los JOINS de JOINED o los UNION ALL de TABLE_PER_CLASS.
 - *Razón 2 — Las consultas polimórficas son frecuentes*
 - El test `getUserByUsername()` devuelve un `User` que puede ser cualquier subtipo. Con SINGLE_TABLE es un SELECT simple. Con las otras dos estrategias implica JOINS o UNION ALL en cada llamada.
 - *Razón 3 — La integridad de subclase se maneja en la capa de negocio*

- La única desventaja real de SINGLE_TABLE es que `expedient` y `education` no pueden ser `NOT NULL` en la BD. Pero mirando los tests, esa validación ya se hace en el servicio: `createDriverUser()` recibe `expedient` como parámetro obligatorio. La integridad está garantizada por código, no por la BD.
- *Razón 4 — TABLE_PER_CLASS queda descartada por el UNION ALL*
 - El test `getUserByUsername()` se llama constantemente. Con TABLE_PER_CLASS cada llamada generaría un UNION ALL sobre tres tablas, lo cual es el peor escenario posible para una operación tan frecuente.

37. Route tiene relaciones muchos-a-muchos con DriverUser y TourGuideUser (subclases de User). Analizar el impacto de la estrategia de herencia sobre la tabla join:

1. Si la jerarquía es SINGLE_TABLE, ¿a que tabla apunta la FK en la tabla join?
2. Si la jerarquía es JOINED, ¿cambia la tabla destino de esa FK?

Si la jerarquía es SINGLE_TABLE, ¿a que tabla apunta la FK en la tabla join?

- Con SINGLE_TABLE existe **una sola tabla** `users` que contiene todos los tipos: `User`, `DriverUser` y `TourGuideUser`. Por lo tanto las FKs de ambas tablas join apuntan siempre a `users`. Ambas FKs (`driver_id` y `tour_guide_id`) apuntan a la **misma tabla** `users`, diferenciándose solo por el valor de `user_type` en la fila destino.

Si la jerarquía es JOINED, ¿cambia la tabla destino de esa FK?

- No, no cambia, sigue apuntando a `users`, la razón es que el `id` es propiedad de `User` y vive en la tabla `users`. Las tablas `driver_users` y `tour_guide_users` no tienen IDs propios, solo tienen una FK hacia `users.id`. Por eso la tabla join referencia `users` directamente.

38. ¿Qué estrategia resulta más robusta ante cambios futuros como agregar una nueva subclase de User? Justificar con al menos dos argumentos.

La estrategia más robusta ante cambios futuros es **JOINED**, por estas razones:

- *Argumento 1 — Agregar una subclase no impacta las tablas existentes*
 - Con **JOINED**, agregar por ejemplo un `AdminUser` con atributos `accessLevel` y `department` solo requiere crear una nueva tabla.
- *Argumento 2 — Mantiene la integridad referencial por diseño*
 - Con **JOINED**, cada tabla de subclase puede tener sus propias restricciones.

Sección 3 - Repositorios, Consultas y Transacciones

3.1 Patrón de acceso a datos: DAO y Repository

39. Definir el patrón DAO (Data Access Object) y el patrón Repository. ¿Cuál es la diferencia conceptual más importante entre ambos? ¿En qué se diferencia su rol dentro de la arquitectura de la aplicación?

Patrón DAO (Data Access Object)

- El **DAO** es un patrón cuyo objetivo es **encapsular la lógica de acceso a datos** de una tabla o entidad específica de la base de datos.
 - **Foco:** Está centrado en los datos y suele mapearse de forma casi directa a las tablas de la base de datos.
 - **Protocolo:** Tradicionalmente implementa todas las operaciones **CRUD** (Create, Read, Update, Delete) de forma explícita. Por ejemplo, un `PersonDAO` tendría un método `update(id, ...)` que recibe datos sueltos para actualizar una fila.
 - **Crítica:** Las fuentes señalan que el DAO suele generar diseños "débiles" u orientados a procedimientos, ya que a menudo le quita a la clase de negocio (como `Person`) la responsabilidad de modificarse a sí misma, delegándola en el DAO.

Patrón Repository

- El **Repository** es un patrón (popularizado por *Domain-Driven Design* o DDD) que busca **simular una colección de objetos en memoria**, ocultando el hecho de que estos objetos están realmente persistidos en una base de datos.
 - **Foco:** Está centrado en el **dominio** y en el comportamiento de una colección clásica (agregar, remover, buscar).
 - **Uso con ORM:** En contextos de Hibernate o JDO, el Repositorio suele concentrar únicamente la operación de **Recuperación (R)** o búsqueda eficiente (ej. `findByName`), ya que la creación y actualización se resuelven de forma transparente mediante la **persistencia por alcance**.
 - **Implementación:** Actúa como una "máscara" que oculta el envío de consultas (HQL/JPQL) a la base de datos.

Diferencia Conceptual más importante

- La diferencia radica en el **nivel de abstracción** y el **modelo mental**:
 - El **DAO** se ve como un adaptador hacia la base de datos (una abstracción sobre el **almacenamiento**). Se piensa en términos de "filas y tablas".
 - El **Repository** se ve como una **colección de objetos de negocio** (una abstracción del **dominio**). Se piensa en términos de "objetos en memoria" aunque por debajo interactúe con la base de datos.

Roles en la Arquitectura de la Aplicación

Característica	Rol del DAO	Rol del Repositorio
Ubicación	Capa de Infraestructura / Datos.	Interfaz en la capa de Dominio, implementación en Infraestructura.
Relación con Entidades	Suele haber un DAO por tabla; puede "ensuciar" el modelo con lógica técnica.	Se crea un repositorio por cada entidad raíz o agregada (ej. <code>PurchaseRepository</code>).
Responsabilidad	Realizar el mapeo físico y ejecutar sentencias de persistencia explícitas.	Concentrar la lógica de búsqueda y recuperación compleja, interactuando con la <code>Session</code> de Hibernate.
Coordinación	El Servicio lo llama para ejecutar cada paso del CRUD manualmente.	El Servicio lo usa para obtener objetos y luego opera sobre ellos, dejando que el ORM sincronice cambios.

40. El patrón Repository puede pensarse como una colección de objetos en memoria que además persiste su estado. Describir cómo se implementa este concepto usando Hibernate: ¿qué responsabilidades concentra un repositorio? ¿Con qué objeto de Hibernate interactúa internamente?

El patrón **Repository** se implementa en Hibernate como una capa de abstracción que busca ocultar la complejidad del acceso a datos, simulando ser una **colección de objetos en memoria** aunque, por debajo, los datos residan en una base de datos relacional.

Responsabilidades de un Repository: En una arquitectura moderna con Hibernate, el repositorio concentra las siguientes tareas:

- **Gestión de la Recuperación (Retrieve):** Su responsabilidad principal es la **búsqueda eficiente** de objetos (por ID, nombre, o criterios complejos). Actúa como una "máscara" que oculta el envío de consultas (HQL/JPQL) a la base de datos.
- **Encapsulamiento Técnico:** Centraliza toda la lógica de acceso a datos, evitando que las capas superiores (como Servicios o el Modelo) se "contaminen" con código SQL o detalles específicos del framework de persistencia.
- **Simulación de Colecciones:** Provee una interfaz que permite al desarrollador tratar a la base de datos como si fuera una colección estándar de Java, permitiendo operaciones como "agregar" o "buscar".
- **Simplificación del protocolo CRUD:** Gracias a la **persistencia por alcance**, muchas veces los métodos `save()` o `update()` no necesitan lógica compleja en el repositorio, ya que Hibernate sincroniza los cambios automáticamente; por ello, el repositorio suele enfocarse casi exclusivamente en la **Recuperación (R)**.

Interacción Interna con Hibernate: Para realizar su trabajo, el repositorio interactúa internamente con el objeto **Session** de Hibernate.

- **Obtención de la Sesión:** El repositorio debe obtener la sesión activa mediante el método `getCurrentSession()` de la **SessionFactory**.
 - **Contexto de Persistencia:** Al interactuar con la `Session`, el repositorio utiliza la **Caché de Nivel 1**, asegurando que los objetos recuperados estén "gestionados" (*managed*) y que cualquier cambio posterior sea detectado por el motor de Hibernate.
 - **Ejecución de Consultas:** El repositorio utiliza la `Session` para crear y ejecutar objetos de tipo **Query** (usando HQL o Criteria), transformando las peticiones de la aplicación en sentencias SQL nativas para el motor de base de datos.
41. Implementar un repositorio por cada entidad del modelo (`PurchaseRepository`, `RouteRepository`, `UserRepository`, `ServiceRepository`, `SupplierRepository`, `ReviewRepository`). Cada repositorio debe incluir al menos las operaciones básicas: guardar, buscar por ID, listar todos y eliminar. Utilizar la `Session` de Hibernate en cada implementación, tal como se ha explicado previamente utilizando el método `getCurrentSession()` sobre la `SessionFactory` (genere una inyección de este objeto tal como se muestra en el código de ejemplo proporcionado)

BaseRepository

```
package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;

public interface BaseRepository<T, ID> {
    T save(T entity);
    Optional<T> findById(ID id);
    List<T> findAll();
    void delete(T entity);
}
```

UserRepository

```
package unlp.info.bd2.repositories;

import java.util.Optional;

import unlp.info.bd2.model.User;

public interface UserRepository extends BaseRepository<User, Long> {
```

```
        Optional<User> findByUsername(String username);  
    }  
}
```

```
package unlp.info.bd2.repositories;  
  
import java.util.List;  
import java.util.Optional;  
import org.hibernate.SessionFactory;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Repository;  
import unlp.info.bd2.model.User;  
  
@Repository  
public class UserRepositoryImpl implements UserRepository {  
  
    @Autowired  
    private SessionFactory sessionFactory;  
  
    @Override  
    public User save(User user) {  
        sessionFactory.getCurrentSession().persist(user);  
        return user;  
    }  
  
    @Override  
    public Optional<User> findById(Long id) {  
        User user = sessionFactory.getCurrentSession().get(User.class, id);  
        return Optional.ofNullable(user);  
    }  
  
    @Override  
    public List<User> findAll() {  
        return sessionFactory.getCurrentSession()  
            .createQuery("FROM User", User.class)  
            .list();  
    }  
  
    @Override  
    public void delete(User user) {  
        sessionFactory.getCurrentSession().remove(user);  
    }  
  
    @Override  
    public Optional<User> findByUsername(String username) {  
        User user = sessionFactory.getCurrentSession()  
            .createQuery("FROM User WHERE username = :username", User.class)  
            .setParameter("username", username)  
            .list().get(0);  
        return Optional.ofNullable(user);  
    }  
}
```

```

        .createQuery("FROM User u WHERE u.username = :username",
User.class)
        .setParameter("username", username)
        .uniqueResult();
    return Optional.ofNullable(user);
}

}

```

RouteRepository

```

package unlp.info.bd2.repositories;

import java.util.List;

import unlp.info.bd2.model.Route;

public interface RouteRepository extends BaseRepository<Route, Long> {
    List<Route> findBelowPrice(float price);
}

```

```

package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import unlp.info.bd2.model.Route;

@Repository
public class RouteRepositoryImpl implements RouteRepository {

    @Autowired
    private SessionFactory sessionFactory;

    @Override
    public Route save(Route route) {
        sessionFactory.getCurrentSession().persist(route);
        return route;
    }

    @Override
    public Optional<Route> findById(Long id) {
        Route route = sessionFactory.getCurrentSession().get(Route.class,

```

```

id);
        return Optional.ofNullable(route);
    }

    @Override
    public List<Route> findAll() {
        return sessionFactory.getCurrentSession()
            .createQuery("FROM Route", Route.class)
            .list();
    }

    @Override
    public void delete(Route route) {
        sessionFactory.getCurrentSession().remove(route);
    }

    @Override
    public List<Route> findBelowPrice(float price) {
        return sessionFactory.getCurrentSession()
            .createQuery("FROM Route r WHERE r.price < :price", Route.class)
            .setParameter("price", price)
            .list();
    }
}

```

PurchaseRepository

```

package unlp.info.bd2.repositories;

import java.util.Optional;

import unlp.info.bd2.model.Purchase;

public interface PurchaseRepository extends BaseRepository<Purchase, Long> {
    Optional<Purchase> findByCode(String code);
}

```

```

package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

```

```

import unlp.info.bd2.model.Purchase;

@Repository
public class PurchaseRepositoryImpl implements PurchaseRepository {

    @Autowired
    private SessionFactory sessionFactory;

    @Override
    public Purchase save(Purchase purchase) {
        sessionFactory.getCurrentSession().persist(purchase);
        return purchase;
    }

    @Override
    public Optional<Purchase> findById(Long id) {
        Purchase purchase =
sessionFactory.getCurrentSession().get(Purchase.class, id);
        return Optional.ofNullable(purchase);
    }

    @Override
    public List<Purchase> findAll() {
        return sessionFactory.getCurrentSession()
            .createQuery("FROM Purchase", Purchase.class)
            .list();
    }

    @Override
    public void delete(Purchase purchase) {
        sessionFactory.getCurrentSession().remove(purchase);
    }

    @Override
    public Optional<Purchase> findByCode(String code) {
        Purchase purchase = sessionFactory.getCurrentSession()
            .createQuery("FROM Purchase p WHERE p.code = :code",
Purchase.class)
            .setParameter("code", code)
            .uniqueResult();
        return Optional.ofNullable(purchase);
    }
}

```

```

package unlp.info.bd2.repositories;

import java.util.Optional;

import unlp.info.bd2.model.Service;

public interface ServiceRepository extends BaseRepository<Service, Long> {
    Optional<Service> findByNameAndSupplierId(String name, Long supplierId);
}

```

```

package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import unlp.info.bd2.model.Service;

@Repository
public class ServiceRepositoryImpl implements ServiceRepository {

    @Autowired
    private SessionFactory sessionFactory;

    @Override
    public Service save(Service service) {
        sessionFactory.getCurrentSession().persist(service);
        return service;
    }

    @Override
    public Optional<Service> findById(Long id) {
        Service service =
sessionFactory.getCurrentSession().get(Service.class, id);
        return Optional.ofNullable(service);
    }

    @Override
    public List<Service> findAll() {
        return sessionFactory.getCurrentSession()
            .createQuery("FROM Service", Service.class)
            .list();
    }
}

```

```

@Override
public void delete(Service service) {
    sessionFactory.getCurrentSession().remove(service);
}

@Override
public Optional<Service> findByNameAndSupplierId(String name, Long
supplierId) {
    Service service = sessionFactory.getCurrentSession()
        .createQuery(
            "FROM Service s WHERE s.name = :name " +
            "AND s.supplier.id = :supplierId", Service.class)
        .setParameter("name", name)
        .setParameter("supplierId", supplierId)
        .uniqueResult();
    return Optional.ofNullable(service);
}
}

```

SupplierRepository

```

package unlp.info.bd2.repositories;

import java.util.Optional;

import unlp.info.bd2.model.Supplier;

public interface SupplierRepository extends BaseRepository<Supplier, Long> {
    Optional<Supplier> findByAuthorizationNumber(String
authorizationNumber);
}

```

```

package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import unlp.info.bd2.model.Supplier;

@Repository
public class SupplierRepositoryImpl implements SupplierRepository {

```

```

@Autowired
private SessionFactory sessionFactory;

@Override
public Supplier save(Supplier supplier) {
    sessionFactory.getCurrentSession().persist(supplier);
    return supplier;
}

@Override
public Optional<Supplier> findById(Long id) {
    Supplier supplier =
sessionFactory.getCurrentSession().get(Supplier.class, id);
    return Optional.ofNullable(supplier);
}

@Override
public List<Supplier> findAll() {
    return sessionFactory.getCurrentSession()
        .createQuery("FROM Supplier", Supplier.class)
        .list();
}

@Override
public void delete(Supplier supplier) {
    sessionFactory.getCurrentSession().remove(supplier);
}

@Override
public Optional<Supplier> findByAuthorizationNumber(String
authorizationNumber) {
    Supplier supplier = sessionFactory.getCurrentSession()
        .createQuery(
            "FROM Supplier s WHERE s.authorizationNumber = :authNum",
            Supplier.class)
        .setParameter("authNum", authorizationNumber)
        .uniqueResult();
    return Optional.ofNullable(supplier);
}
}

```

ReviewRepository

```

package unlp.info.bd2.repositories;

```

```
import unlp.info.bd2.model.Review;

public interface ReviewRepository extends BaseRepository<Review, Long> {

}
```

```
package unlp.info.bd2.repositories;

import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import unlp.info.bd2.model.Review;

import java.util.List;
import java.util.Optional;

@Repository
public class ReviewRepositoryImpl implements ReviewRepository {

    @Autowired
    private SessionFactory sessionFactory;

    @Override
    public Review save(Review review) {
        sessionFactory.getCurrentSession().persist(review);
        return review;
    }

    @Override
    public Optional<Review> findById(Long id) {
        Review review = sessionFactory.getCurrentSession().get(Review.class, id);
        return Optional.ofNullable(review);
    }

    @Override
    public List<Review> findAll() {
        return sessionFactory.getCurrentSession()
            .createQuery("FROM Review", Review.class)
            .list();
    }

    @Override
    public void delete(Review review) {
        sessionFactory.getCurrentSession().remove(review);
    }
}
```

```
}
```

```
}
```

3.2 Transacciones

42. ¿Que es una transacción en el contexto de Hibernate? ¿Por qué es necesaria? ¿Qué ocurre si se realizan operaciones de escritura sin una transacción activa?

Definición

- Una **transacción** se define como la delimitación de una **unidad de trabajo**. Es un conjunto de operaciones (mensajes enviados a los objetos) que deben ejecutarse bajo las propiedades **ACID**: Atómica, Consistente, Aislada y Duradera.
 - En Hibernate, la transacción actúa como un objeto al que se le envían mensajes como `begin()`, `commit()` y `rollback()` para controlar el flujo de persistencia.

¿Por qué es necesaria?

- **Gestión de la Atomicidad**: Asegura que muchas operaciones que afectan a múltiples filas y tablas se realicen como una sola acción. Si una falla, se deshace todo el conjunto para no dejar los datos a medias.
- **Control de Concurrencia**: En aplicaciones multiusuario, evita conflictos (como el "doble registro" de una plaza en un tour) al coordinar el acceso a los mismos recursos.
- **Consistencia Lógica**: Garantiza que los diferentes ítems de datos tengan sentido entre sí (por ejemplo, que el monto total de una `Purchase` coincida con la suma de sus `ItemService`).
- **Recuperación de Errores**: Permite realizar un **rollback** para limpiar el estado de la aplicación si ocurre un error durante el procesamiento de un cambio.
- **Soporte de Distribución**: Es la solución técnica para evitar que múltiples servidores toquen el mismo recurso físico al mismo tiempo.

¿Qué ocurre si no hay una transacción activa? Realizar operaciones de escritura sin una transacción demarcada conlleva graves riesgos y comportamientos técnicos específicos:

- **Falta de Durabilidad**: Los cambios realizados en los objetos en memoria podrían **no persistirse nunca físicamente** en la base de datos, ya que Hibernate utiliza el final de la transacción (`commit`) para realizar el **flush** (sincronización) de los cambios.
- **Inconsistencia de Datos**: El sistema queda expuesto a un "período de inconsistencia" donde otros clientes podrían leer datos parciales o erróneos.
- **Excepciones del Framework**: Dependiendo de la configuración, Hibernate puede lanzar excepciones (como `TransactionRequiredException` en JPA o errores al intentar usar

`getCurrentSession()`) indicando que el contexto de persistencia requiere una transacción activa para operar.

- **Pérdida de Aislamiento:** Sin una transacción, no hay garantía de que los cambios de un usuario estén aislados de los demás, lo que invalida las garantías de seguridad de la base de datos relacional.
43. ¿En qué capa de la aplicación debería gestionarse la transacción: en el repositorio o en la capa de servicio? Justificar la elección. ¿Qué ocurre si una misma operación necesita de varios accesos a la base de datos?

La transacción debe gestionarse en la capa de servicio, no en el repositorio. Justificación:

- *Razón 1 — La unidad de trabajo es de negocio, no de acceso a datos*
 - Una transacción delimita una **unidad de trabajo con sentido para el negocio**. El repositorio no sabe qué constituye una operación completa — solo sabe persistir o recuperar objetos individuales. Es el servicio quien conoce qué conjunto de operaciones debe ejecutarse de forma atómica.
 - *Razón 2 — El repositorio debe ser reutilizable y simple*
 - Si el repositorio gestiona su propia transacción, cada llamada abre y cierra una transacción independiente.
44. Implementar una capa de servicio que coordine las operaciones del modelo usando transacciones correctamente. Como mínimo implementar:
1. Creación de todas las entidades persistentes.
 2. Actualización del precio de un servicio existente.
 3. Agregar un nuevo ítem a una compra existente.
 4. Eliminar una ruta existente siempre que no tenga compras asociadas.

ToursServiceImpl

```
package unlp.info.bd2.services;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import unlp.info.bd2.model.DriverUser;
import unlp.info.bd2.model.ItemService;
```

```
import unlp.info.bd2.model.Purchase;
import unlp.info.bd2.model.Review;
import unlp.info.bd2.model.Route;
import unlp.info.bd2.model.Stop;
import unlp.info.bd2.model.Supplier;
import unlp.info.bd2.model.TourGuideUser;
import unlp.info.bd2.model.User;
import unlp.info.bd2.repositories.ItemServiceRepository;
import unlp.info.bd2.repositories.PurchaseRepository;
import unlp.info.bd2.repositories.ReviewRepository;
import unlp.info.bd2.repositories.RouteRepository;
import unlp.info.bd2.repositories.ServiceRepository;
import unlp.info.bd2.repositories.StopRepository;
import unlp.info.bd2.repositories.SupplierRepository;
import unlp.info.bd2.repositories.UserRepository;
import unlp.info.bd2.utils.ToursException;
```

```
@Service
```

```
@Transactional
```

```
public class ToursServiceImpl implements ToursService {
```

```
    @Autowired
```

```
    private UserRepository userRepository;
```

```
    @Autowired
```

```
    private RouteRepository routeRepository;
```

```
    @Autowired
```

```
    private PurchaseRepository purchaseRepository;
```

```
    @Autowired
```

```
    private ServiceRepository serviceRepository;
```

```
    @Autowired
```

```
    private SupplierRepository supplierRepository;
```

```
    @Autowired
```

```
    private ReviewRepository reviewRepository;
```

```
    @Autowired
```

```
    private StopRepository stopRepository;
```

```
    @Autowired
```

```
    private ItemServiceRepository itemServiceRepository;
```

```
// dejo este constructor vacío para Spring porque lo usamos la practica
```

```

que sigue
    public ToursServiceImpl() {}

    // — USER —————
    @Override
    public User createUser(String username, String password, String
fullName,
        String email, Date birthdate, String phoneNumber) throws
ToursException {
        try {
            User user = new User();
            user.setUsername(username);
            user.setPassword(password);
            user.setName(fullName);
            user.setEmail(email);
            user.setBirthdate(birthdate);
            user.setPhoneNumber(phoneNumber);
            user.setActive(true);
            return userRepository.save(user);
        } catch (Exception e) {
            throw new ToursException("Constraint Violation");
        }
    }

    @Override
    public DriverUser createDriverUser(String username, String password,
        String fullName, String email, Date birthdate,
        String phoneNumber, String expedient) throws ToursException {
        try {
            DriverUser driver = new DriverUser();
            driver.setUsername(username);
            driver.setPassword(password);
            driver.setName(fullName);
            driver.setEmail(email);
            driver.setBirthdate(birthdate);
            driver.setPhoneNumber(phoneNumber);
            driver.setExpedient(expedient);
            driver.setActive(true);
            return (DriverUser) userRepository.save(driver);
        } catch (Exception e) {
            throw new ToursException("Constraint Violation");
        }
    }

    @Override
    public TourGuideUser createTourGuideUser(String username, String

```

```

password,
        String fullName, String email, Date birthdate,
        String phoneNumber, String education) throws ToursException {
    try {
        TourGuideUser guide = new TourGuideUser();
        guide.setUsername(username);
        guide.setPassword(password);
        guide.setName(fullName);
        guide.setEmail(email);
        guide.setBirthdate(birthdate);
        guide.setPhoneNumber(phoneNumber);
        guide.setEducation(education);
        guide.setActive(true);
        return (TourGuideUser) userRepository.save(guide);
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
@Transactional(readOnly = true)
public Optional<User> getUserById(Long id) throws ToursException {
    return userRepository.findById(id);
}

@Override
@Transactional(readOnly = true)
public Optional<User> getUserByUsername(String username) throws
ToursException {
    return userRepository.findByUsername(username);
}

@Override
public User updateUser(User user) throws ToursException {
    Optional<User> original = userRepository.findById(user.getId());
    if (original.isEmpty()) throw new ToursException("Usuario no
encontrado");
    User existing = original.get();
    existing.setPassword(user.getPassword());
    existing.setName(user.getName());
    existing.setEmail(user.getEmail());
    existing.setBirthdate(user.getBirthdate());
    existing.setPhoneNumber(user.getPhoneNumber());
    if (user instanceof DriverUser && existing instanceof DriverUser) {
        ((DriverUser) existing).setExpedient(((DriverUser)
user).getExpedient());
    }
}

```

```

    }
    if (user instanceof TourGuideUser && existing instanceof
TourGuideUser) {
        ((TourGuideUser) existing).setEducation(((TourGuideUser)
user).getEducation());
    }
    // existing está en estado managed → Hibernate detecta cambios
    // y los sincroniza en el flush automáticamente
    return existing;
}

@Override
public void deleteUser(User user) throws ToursException {
    if (!user.isActive()) {
        throw new ToursException("El usuario se encuentra desactivado");
    }
    if (user instanceof TourGuideUser) {
        TourGuideUser guide = (TourGuideUser) user;
        if (guide.getRoutes() != null && !guide.getRoutes().isEmpty()) {
            throw new ToursException("El usuario no puede ser
desactivado");
        }
    }
    if (user.getPurchaseList() != null &&
!user.getPurchaseList().isEmpty()) {
        user.setActive(false);
        // managed → Hibernate sincroniza el cambio solo
    } else {
        userRepository.delete(user);
    }
}

// — STOP —————

@Override
public Stop createStop(String name, String description) throws
ToursException {
    Stop stop = new Stop();
    stop.setName(name);
    stop.setDescription(description);
    // necesitamos un StopRepository — lo agregamos abajo
    return stopRepository.save(stop);
}

@Override
@Transactional(readOnly = true)

```

```

public List<Stop> getStopByNameStart(String name) {
    return stopRepository.findByNameStart(name);
}

// — ROUTE —————

@Override
public Route createRoute(String name, float price, float totalKm,
    int maxNumberOfUsers, List<Stop> stops) throws ToursException {
    Route route = new Route();
    route.setName(name);
    route.setPrice(price);
    route.setTotalKm(totalKm);
    route.setMaxNumberUsers(maxNumberOfUsers);
    route.setStops(stops);
    return routeRepository.save(route);
}

@Override
@Transactional(readOnly = true)
public Optional<Route> getRouteById(Long id) {
    return routeRepository.findById(id);
}

@Override
@Transactional(readOnly = true)
public List<Route> getRoutesBelowPrice(float price) {
    return routeRepository.findBelowPrice(price);
}

@Override
public void assignDriverByUsername(String username, Long idRoute) throws
ToursException {
    Optional<User> optUser = userRepository.findByUsername(username);
    if (optUser.isEmpty() || !(optUser.get() instanceof DriverUser)) {
        throw new ToursException("No pudo realizarse la asignación");
    }
    Optional<Route> optRoute = routeRepository.findById(idRoute);
    if (optRoute.isEmpty()) {
        throw new ToursException("No pudo realizarse la asignación");
    }
    optRoute.get().getDriverList().add((DriverUser) optUser.get());
    // route está managed → Hibernate sincroniza la tabla join
}

@Override

```

```

    public void assignTourGuideByUsername(String username, Long idRoute)
throws ToursException {
    Optional<User> optUser = userRepository.findByUsername(username);
    if (optUser.isEmpty() || !(optUser.get() instanceof TourGuideUser))
{
        throw new ToursException("No pudo realizarse la asignación");
    }
    Optional<Route> optRoute = routeRepository.findById(idRoute);
    if (optRoute.isEmpty()) {
        throw new ToursException("No pudo realizarse la asignación");
    }
    optRoute.get().getTourGuideList().add((TourGuideUser)
optUser.get());
}

```

@Override

```

public void deleteRoute(Route route) throws ToursException {
    long count = routeRepository.countPurchasesByRoute(route.getId());
    if (count > 0) {
        throw new ToursException(
            "No puede eliminarse una ruta con compras asociadas");
    }
    routeRepository.delete(route);
}

```

// — SUPPLIER —————

@Override

```

public Supplier createSupplier(String businessName,
    String authorizationNumber) throws ToursException {
    try {
        Supplier supplier = new Supplier();
        supplier.setBusinessName(businessName);
        supplier.setAuthorizationNumber(authorizationNumber);
        return supplierRepository.save(supplier);
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

```

@Override

```

@Transactional(readOnly = true)
public Optional<Supplier> getSupplierById(Long id) {
    return supplierRepository.findById(id);
}

```

```

@Override
@Transactional(readonly = true)
public Optional<Supplier> getSupplierByAuthorizationNumber(String
authorizationNumber) {
    return
supplierRepository.findByAuthorizationNumber(authorizationNumber);
}

// — SERVICE —————

@Override
public unlp.info.bd2.model.Service addServiceToSupplier(String name,
float price,
    String description, Supplier supplier) throws ToursException {
    unlp.info.bd2.model.Service service = new
unlp.info.bd2.model.Service();
    service.setName(name);
    service.setPrice(price);
    service.setDescription(description);
    service.setSupplier(supplier);
    if (supplier.getServices() == null) {
        supplier.setServices(new ArrayList<>());
    }
    supplier.getServices().add(service);
    return serviceRepository.save(service);
}

@Override
public unlp.info.bd2.model.Service updateServicePriceById(Long id, float
newPrice) throws ToursException {
    Optional<unlp.info.bd2.model.Service> optService =
serviceRepository.findById(id);
    if (optService.isEmpty()) {
        throw new ToursException("No existe el producto");
    }
    unlp.info.bd2.model.Service service = optService.get();
    service.setPrice(newPrice);
    // managed → Hibernate detecta el cambio y lo sincroniza
    return service;
}

@Override
@Transactional(readonly = true)
public Optional<unlp.info.bd2.model.Service>
getServiceByNameAndSupplierId(String name, Long id) throws ToursException {
    return serviceRepository.findByNameAndSupplierId(name, id);
}

```

```

}

// — PURCHASE —————

@Override
public Purchase createPurchase(String code, Route route,
    User user) throws ToursException {
    return createPurchase(code, new Date(), route, user);
}

@Override
public Purchase createPurchase(String code, Date date, Route route,
    User user) throws ToursException {
    long count = routeRepository.countPurchasesByRoute(route.getId());
    if (count ≥ route.getMaxNumberUsers()) {
        throw new ToursException("No puede realizarse la compra");
    }
    try {
        Purchase purchase = new Purchase();
        purchase.setCode(code);
        purchase.setDate(date);
        purchase.setRoute(route);
        purchase.setUser(user);
        purchase.setTotalPrice(route.getPrice());
        if (user.getPurchaseList() == null) {
            user.setPurchaseList(new ArrayList<>());
        }
        user.getPurchaseList().add(purchase);
        return purchaseRepository.save(purchase);
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
public ItemService addItemToPurchase(unlp.info.bd2.model.Service
service, int quantity,
    Purchase purchase) throws ToursException {
    ItemService item = new ItemService();
    item.setService(service);
    item.setQuantity(quantity);
    purchase.addItem(item);
    float newTotal = purchase.getTotalPrice() + (service.getPrice() *
quantity);
    purchase.setTotalPrice(newTotal);
    // purchase está managed → el cambio en totalPrice se sincroniza

```

```

        return itemServiceRepository.save(item);
    }

    @Override
    @Transactional(readOnly = true)
    public Optional<Purchase> getPurchaseByCode(String code) {
        return purchaseRepository.findByCode(code);
    }

    @Override
    public void deletePurchase(Purchase purchase) throws ToursException {
        purchaseRepository.delete(purchase);
    }

    // — REVIEW —————

    @Override
    public Review addReviewToPurchase(int rating, String comment,
        Purchase purchase) throws ToursException {
        Review review = new Review();
        review.setRating(rating);
        review.setComment(comment);
        review.setPurchase(purchase);
        purchase.setReview(review);
        return reviewRepository.save(review);
    }

    // — HQL (después) —————
    @Override public List<Purchase> getAllPurchasesOfUsername(String
username) { return null; }
    @Override public List<User> getUserSpendingMoreThan(float mount) {
return null; }
    @Override public List<Supplier> getTopNSuppliersInPurchases(int n) {
return null; }
    @Override public long getCountOfPurchasesBetweenDates(Date start, Date
end) { return 0; }
    @Override public List<Route> getRoutesWithStop(Stop stop) { return null;
}
    @Override public Long getMaxStopOfRoutes() { return null; }
    @Override public List<Route> getRoutsNotSell() { return null; }
    @Override public List<Route> getTop3RoutesWithMaxRating() { return null;
}
    @Override public unlp.info.bd2.model.Service getMostDemandedService() {
return null; }
    @Override public List<TourGuideUser> getTourGuidesWithRating1() { return
null; }

```

```
}
```

3.3 Consultas con HQL y JPQL

45. ¿Que diferencia hay entre HQL/JPQL y SQL nativo? ¿Qué entidades, atributos y relaciones entienden HQL/JPQL que SQL no conoce directamente?

Diferencia entre HQL/JPQL y SQL

- **SQL Nativo:** Está centrado en el **modelo relacional**. Las consultas se realizan sobre **tablas y columnas**. El resultado de una consulta SQL es un conjunto de tuplas (filas) de datos planos.
- **HQL / JPQL:** Está centrado en el **modelo de objetos**. Las consultas se realizan sobre **clases y sus atributos**. Hibernate traduce estas consultas al SQL específico (dialecto) del motor de base de datos subyacente para su ejecución. El resultado suele ser una colección de **objetos gestionados** (*Managed*) que conservan su identidad y estado dentro del Persistence Context.

Conceptos que HQL/JPQL entiende (y SQL no conoce directamente): HQL y JPQL incorporan semántica propia del paradigma orientado a objetos que SQL, al ser "ignorante de los objetos", no puede procesar sin transformaciones manuales complejas:

1. Jerarquías y Polimorfismo:

- HQL entiende la **herencia**. Si se realiza una consulta sobre una superclase (ej. `FROM User`), Hibernate automáticamente incluye en los resultados a todas las instancias de sus subclases (`DriverUser`, `TourGuideUser`).
- En SQL nativo, el motor no sabe que una tabla es "hija" de otra en términos de comportamiento; el desarrollador debe realizar manualmente los `JOINS` o `UNIONS` necesarios según la estrategia de mapeo utilizada.

2. Navegación de Asociaciones (Path Expressions):

- Permiten navegar por las relaciones del modelo utilizando el **operador punto** (`.`), por ejemplo: `SELECT p.route.name FROM Purchase p`.
- SQL nativo requiere que el desarrollador especifique explícitamente las cláusulas `JOIN` y las condiciones de igualdad entre claves foráneas y primarias para saltar de una tabla a otra.

3. Colecciones:

- HQL comprende que un atributo de una clase puede ser una **colección de objetos** (como un `Set` o `List`). Permite realizar joins directamente sobre estos atributos: `JOIN p.itemsServiceList item`.
- Para SQL, las colecciones no existen como tales dentro de una fila; son simplemente registros en otras tablas vinculados por una clave, perdiendo la semántica de

"contenedor" que tiene el objeto.

4. Tipos de Datos y Mapeos:

- HQL opera sobre los tipos de datos de Java (ej. `Enum`, `Date`, `String`) y conoce cómo se mapean a los tipos específicos del motor (ej. `VARCHAR2`, `TIMESTAMP`).
- SQL nativo está atado estrictamente a los tipos definidos en el esquema físico de la base de datos.

5. Implementar las siguientes consultas en los repositorios correspondientes:

1. List `getAllPurchasesOfUsername(String username)` Obtiene y retorna el listado de compras realizadas por el usuario con el nombre de usuario especificado por parámetro.
2. List `getUserSpendingMoreThan(float mount)`; Obtiene y retorna el listado de usuarios que hayan efectuado alguna compra por un valor mayor o igual al pasado por parámetro.
3. List `getTopNSuppliersInPurchases(int n)`; Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de apariciones en compras.
4. int `getCountOfPurchasesBetweenDates (Date start, Date end)`; Obtiene y retorna la cantidad de compras registradas entre dos fechas.
5. List `getRoutesWithStop(Stop stop)`; Obtiene y retorna el listado de rutas que incluyen una parada stop especificado por parámetro.
6. int `getMaxStopOfRoutes()`; Obtiene y retorna la cantidad de paradas que posee el recorrido con mayor cantidad de stops.
7. List `getRoutesNotSell()`; Obtiene y retorna el listado de recorridos que no fueron vendidas.
8. List `getTop3RoutesWithMaxRating()`; Obtiene y retorna los tres recorridos con mayor promedio de rating en las reviews de compras asociadas a dicha ruta.
9. Service `getMostDemandedService()`; Obtiene y retorna el servicio que más veces fue incluido en compras, teniendo en cuenta la cantidad.
10. List `getTourGuidesWithRating1()`; Obtiene y retorna el listado de guías asignados a algún tour con un rating de una estrella.

Dónde va cada consulta

Consulta	Repositorio
<code>getAllPurchasesOfUsername</code>	<code>PurchaseRepository</code>
<code>getUserSpendingMoreThan</code>	<code>UserRepository</code>
<code>getTopNSuppliersInPurchases</code>	<code>SupplierRepository</code>
<code>getCountOfPurchasesBetweenDates</code>	<code>PurchaseRepository</code>
<code>getRoutesWithStop</code>	<code>RouteRepository</code>

Consulta	Repositorio
getMaxStopOfRoutes	RouteRepository
getRoutesNotSell	RouteRepository
getTop3RoutesWithMaxRating	RouteRepository
getMostDemandedService	ServiceRepository
getTourGuidesWithRating1	UserRepository

Todas las consultas

```

@Override
public List<Purchase> getAllPurchasesOfUsername(String username) {
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT p FROM Purchase p " +
            "WHERE p.user.username = :username", Purchase.class)
        .setParameter("username", username)
        .list();
}

@Override
public long getCountOfPurchasesBetweenDates(Date start, Date end) {
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT COUNT(p) FROM Purchase p " +
            "WHERE p.date ≥ :start AND p.date ≤ :end", Long.class)
        .setParameter("start", start)
        .setParameter("end", end)
        .uniqueResult();
}

@Override
public List<User> getUserSpendingMoreThan(float mount) {
    // busca usuarios que tengan al menos una compra
    // con totalPrice ≥ mount
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT DISTINCT p.user FROM Purchase p " +
            "WHERE p.totalPrice ≥ :mount", User.class)
        .setParameter("mount", mount)
        .list();
}

@Override

```

```

public List<TourGuideUser> getTourGuidesWithRating1() {
    return this.sessionFactory.getCurrentSession()
        .createQuery("select distinct g from Route r join r.tourGuides g,
Purchase p "
            + "where p.route = r and p.review.rating = 1",
TourGuideUser.class)
        .getResultList();
}

```

```

@Override
public List<Supplier> getTopNSuppliersInPurchases(int n) {
    // navega: Supplier ← Service ← ItemService ← Purchase
    // cuenta cuántas veces aparece cada supplier en compras
    // y ordena de mayor a menor
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT s.supplier FROM Service s " +
            "JOIN s.itemServiceList item " +
            "JOIN item.purchase p " +
            "GROUP BY s.supplier " +
            "ORDER BY COUNT(p) DESC", Supplier.class)
        .setMaxResults(n)
        .list();
}

```

```

@Override
public List<Route> getRoutesWithStop(Stop stop) {
    // rutas que contienen la parada especificada
    // navega la relación ManyToMany Route → stops
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT DISTINCT r FROM Route r " +
            "JOIN r.stops s " +
            "WHERE s = :stop", Route.class)
        .setParameter("stop", stop)
        .list();
}

```

```

@Override
public Long getMaxStopOfRoutes() {
    // cantidad de stops del recorrido con más paradas
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT MAX(SIZE(r.stops)) FROM Route r",
            Long.class)
        .uniqueResult();
}

```

```

}

@Override
public List<Route> getRoutesNotSell() {
    // rutas que no tienen ninguna compra asociada
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT r FROM Route r " +
            "WHERE r NOT IN (" +
            "    SELECT DISTINCT p.route FROM Purchase p" +
            ")", Route.class)
        .list();
}

@Override
public List<Route> getTop3RoutesWithMaxRating() {
    return this.sessionFactory.getCurrentSession()
        .createQuery("select p.route from Purchase p where p.review is not
null " +
            "group by p.route order by avg(p.review.rating) desc",
Route.class)
        .setMaxResults(3)
        .getResultList();
}

@Override
public Service getMostDemandedService() {
    // el servicio que más veces fue incluido en compras
    // teniendo en cuenta la cantidad (quantity) de cada ItemService
    return sessionFactory.getCurrentSession()
        .createQuery(
            "SELECT item.service FROM ItemService item " +
            "GROUP BY item.service " +
            "ORDER BY SUM(item.quantity) DESC",
Service.class)
        .setMaxResults(1)
        .uniqueResult();
}

```

47. ¿En qué casos conviene usar una consulta SQL nativa en lugar de HQL/JPQL? Describir al menos un caso concreto del modelo donde esto sería necesario.

Caso 1 — Funciones específicas del motor de base de datos: HQL solo conoce funciones estándar. Si necesitas usar una función propia de MySQL, PostgreSQL u otro motor, HQL no

puede expresarla. **Ejemplo concreto en el modelo:** supongamos que querés obtener las compras del último mes usando la función `DATE_FORMAT` de MySQL:

```
// HQL no puede hacer esto
"SELECT p FROM Purchase p WHERE DATE_FORMAT(p.date, '%Y-%m') =
DATE_FORMAT(NOW(), '%Y-%m')"

// SQL nativo sí puede
sessionFactory.getCurrentSession()
    .createNativeQuery(
        "SELECT * FROM purchases " +
        "WHERE DATE_FORMAT(date, '%Y-%m') = DATE_FORMAT(NOW(), '%Y-%m')",
        Purchase.class)
    .getResultList();
```

Caso 2 — Consultas de alto rendimiento con optimizaciones manuales: Cuando Hibernate genera SQL ineficiente para una consulta compleja y necesitás control total sobre el plan de ejecución: hints, índices específicos, o estructuras que HQL no puede expresar. **Ejemplo concreto:** obtener los proveedores top usando un `STRAIGHT_JOIN` de MySQL para forzar un orden de join específico que el optimizador de Hibernate no elegiría:

```
sessionFactory.getCurrentSession()
    .createNativeQuery(
        "SELECT STRAIGHT_JOIN sup.* FROM suppliers sup " +
        "INNER JOIN services svc ON svc.supplier_id = sup.id " +
        "INNER JOIN item_services item ON item.service_id = svc.id " +
        "GROUP BY sup.id " +
        "ORDER BY COUNT(item.id) DESC " +
        "LIMIT :n",
        Supplier.class)
    .setParameter("n", n)
    .getResultList();
```

Caso 3 — Operaciones masivas sobre la base de datos: Para `UPDATE` o `DELETE` masivos donde HQL generaría múltiples roundtrips por el ciclo de vida de las entidades, SQL nativo opera directamente sobre las filas sin pasar por el Persistence Context:

```
// Dar de baja todos los usuarios inactivos hace más de un año
// HQL cargaría cada entidad en memoria antes de borrarla

// SQL nativo lo hace en una sola operación
sessionFactory.getCurrentSession()
    .createNativeQuery(
```

```
"UPDATE users SET active = false " +  
"WHERE active = true " +  
"AND id NOT IN (SELECT user_id FROM purchases) " +  
"AND birthdate < DATE_SUB(NOW(), INTERVAL 1 YEAR)")  
.executeUpdate();
```